Colegio de Montalban
Kasiglahan Village, San Jose, Rodriguez, Rizal
Institute of Computing Studies

Learning Management System for Institute of Computing Studies at Colegio de Montalban

An Undergraduate Thesis Presented to the

Institute of Computing Studies

Colegio de Montalban

Rodriguez, Rizal

In partial fulfillment of the

requirements for the degree

Bachelor of Science in

Information Technology

Amigo, Neil Kirby S.

Prado, Kirsten Kate D.

Taclendo, Jane Mae L.

Taño, Devi D.

May 2026

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND OF THE STUDY

Learning Management Systems (LMS) support the organization of course materials, academic activities, and communication across different devices. International research shows that their usefulness depends on how well the available functions meet learners' needs. In Saudi Arabia, Alfalah (2023) surveyed 258 students at Hail University, Aljouf University, and Northern Border University about mobile LMS adoption. Performance expectancy, effort expectancy, and learning intention were associated with intention to use the system. These findings connect access with students' expectations of useful and understandable academic functions, providing a broad basis for examining a multi-platform learning environment.

Within computing education, an LMS also needs to support subject-specific activities and purposeful participation. Ahmed et al. (2025) evaluated a gamified, mobile, cloud-based LMS with 60 second-year students taking Object-Oriented Programming at Mansoura University in Egypt. The experimental group obtained higher post-test achievement and engagement scores than the comparison group. Jaramillo-Mediavilla et al. (2024) reviewed gamification research and reported favorable findings in parts of the literature, while identifying gaps in examining motivation and academic performance together. These studies support considering programming and participation tools within an LMS, although their educational results cannot be presumed for another institution.

In the Philippines, institutional studies connect LMS use with course organization and the experience of completing academic tasks. Llamas et al. (2025) examined the Moodle-based LMS at Mindanao State University–Iligan Institute of Technology (MSU–IIT) in Iligan City and identified concerns in usability, communication, course building, monitoring and assessment, content management, and technical support. Dela Rosa (2023) examined Canvas Free for Teachers with third-year BSIT students at Bulacan State University, who evaluated its lesson modules and assessments favorably. Together, these studies indicate that academic functions and user experiences must be examined within their particular institutional and course settings.

At Colegio de Montalban, the present study focuses on the Institute of Computing Studies (ICS), which offers the Bachelor of Science in Information Technology (BSIT) and Bachelor of Science in Computer Engineering (BSCpE) programs. Its computing workflows involve course resources, submissions, programming activities, assessment records, personal study, and faculty coordination. The research group's initial project description identifies the use of separate academic and personal-organization tools as a concern. Local requirements gathering will establish the extent of this fragmentation and the specific difficulties experienced by ICS students and instructors.

The development problem is how to connect these activities while maintaining clear responsibilities and appropriate access to information. Students require understandable access to approved coursework, personal grades, and study tools. Instructors require manageable teaching, grading, and communication workflows. Academic oversight and administration require their own restricted functions, particularly when course information, faculty responsibilities, schedules, and calendar events are shared. These requirements call for a coordinated design that protects individual academic records.

In response, the study proposes the CdM Learning Management System (CdM LMS) for ICS. The Web Portal will serve as the principal academic environment, the Mobile Application will support selected academic and productivity activities, and the native Desktop Application will function as the Administrator Console. Shared backend workflows and a centralized database will connect their applicable records. The researchers will develop and evaluate the system using role-appropriate feedback from ICS students and instructors and researcher-conducted technical checks. The study will provide evidence about the evaluated release's software quality and workflow suitability rather than assume improvements in grades or long-term learning outcomes.

1.2 STATEMENT OF THE PROBLEM

This study seeks to determine the level of software quality of the LMS for ICS at Colegio de Montalban in supporting coordinated academic resource and grade management, learning support, academic coordination, and system administration. The evaluation will follow six selected

characteristics of the ISO/IEC 25010:2023 Software Product Quality Model (International Organization for Standardization [ISO], 2023).

Specifically, the study seeks to answer the following questions:

1. What is the evaluated software quality of CdM LMS in supporting academic resource and grade-management workflows in terms of functional suitability, as measured by:

• Functional completeness in supporting course materials, assignments, dynamic grading, CodeLab activities, Git-integrated submissions, and the defined academic coordination functions;

• Functional correctness of weighted grade computations, EXP and level calculations, descriptive analytics, and schedule revisions and exceptions;

• Functional appropriateness in supporting connected academic workflows for BSIT and BSCpE users within their assigned roles and platform responsibilities?

2. What is the evaluated software quality of CdM LMS in providing understandable and engaging task interaction in terms of interaction capability, as reflected in:

• Appropriateness recognizability of course, grading, productivity, gamification, and academic coordination functions;

• Learnability of permitted tasks in the Web Portal, selected Mobile Application functions, and administrator-only Desktop Application;

• User engagement supported by EXP, levels, achievements, leaderboards, study tools, and the moderated Knowledge Exchange environment?

3. What is the evaluated software quality of CdM LMS in protecting academic information and controlling access in terms of security, as demonstrated by:

• Confidentiality of student grades, personal notes, academic records, and restricted course information;

• Integrity of grades, submissions, schedule records, and events during authorized updates and concurrent actions;

• Accountability through activity and audit records for designated administrative actions, academic coordination changes, and Knowledge Exchange moderation?

4. What is the evaluated software quality of CdM LMS in processing academic information and maintaining applicable shared records in terms of performance efficiency, as indicated by:

• Time behavior in computing grades, generating descriptive analytics and calendar views, retrieving course resources, and displaying relevant in-LMS updates;

• Resource utilization during the processing and retrieval of shared information by the authorized Web, Mobile, and Desktop components under defined workloads?

5. What is the evaluated software quality of CdM LMS in providing stable access to permitted academic and study functions in terms of reliability, as evaluated through:

• Availability of the Web Portal, supported mobile study tools, and Administrator Console during the defined evaluation period;

• Fault tolerance when processing interrupted requests, concurrent grade updates, or simultaneous Knowledge Exchange activity, including the handling of failed operations without incomplete or duplicate records?

6. What is the evaluated software quality of CdM LMS in supporting consistent information exchange and operation alongside other software in terms of compatibility, as indicated by:

• Interoperability among the Web Portal, Mobile Application, and Administrator Console through their applicable shared records and authorized backend workflows;

• Co-existence of the Administrator Console with other software in the supported administrator workstation environment without adversely affecting their operation?

1.3 STATEMENT OF THE OBJECTIVES

1.3.1 General Objective

To develop and evaluate a Learning Management System for the Institute of Computing Studies at Colegio de Montalban that coordinates academic management, learning support, academic oversight, and system administration through shared information and role-appropriate access.

1.3.2 Specific Objectives

1. Establish shared backend services and a centralized PostgreSQL database for the principal Web Portal, complementary Mobile Application, and administrator-only Desktop Application, with credential-based authentication, account-status controls, and server-enforced access for Student, Instructor, Super User, and Administrator roles.

2. Develop course and section management that allows authorized instructors to organize materials, annotations, assignments, announcements, and deadlines, and allows enrolled students to access resources and submit permitted work under the applicable validation rules.

3. Implement a Dynamic Gradebook with configurable assessment components and weights, score recording, automatic weighted-grade computations, personal student grade views, instructor performance summaries, and CSV grade export; verify computed values and charts against documented test records.

4. Provide CodeLab for in-browser programming execution and assessment against predefined test cases, and a separate Git-integrated workflow for repository-based coursework submission and instructor review; verify the supported submission and access rules for each workflow.

5. Develop notes, task lists, flashcards, review and mastery records, focus timers, and study-session tracking, together with rule-based EXP, levels, achievements, badges, streaks, and department leaderboards; verify progression calculations and the separation of EXP from course grades and forum reputation.

6. Implement Knowledge Exchange for academic questions, answers, and resource sharing, with authorized instructor moderation, Faculty-Verified Answers, contribution-based reputation, and records of designated moderation actions.

7. Develop the faculty directory and academic operations hub, distinguishing program affiliations and Dean or Program Head designations from primary account roles, and providing restricted course supervision that excludes student identities, individual grades, submissions, assessment attempts, and answers.

8. Implement Asia/Manila-based weekly course schedules with effective-dated revision history and future occurrence cancellation, rescheduling, and restoration; provide authorized calendar views, audience-targeted saved events, coursework deadlines, and private note–event snapshot conversion, and verify their visibility and preservation rules.

9. Provide the Administrator Console with account approval, status and role management, permitted configuration, activity and audit records, reporting, and the backup and recovery functions supported by the evaluated release; verify that administrative permissions remain separate from academic oversight.

10. Evaluate the developed system in terms of functional suitability, interaction capability, security, performance efficiency, reliability, and compatibility using role-appropriate tasks, feedback from eligible ICS students and instructors, and technical checks conducted by the researchers, and document defects and limitations identified in the evaluated version.

1.4 CONCEPTUAL FRAMEWORK

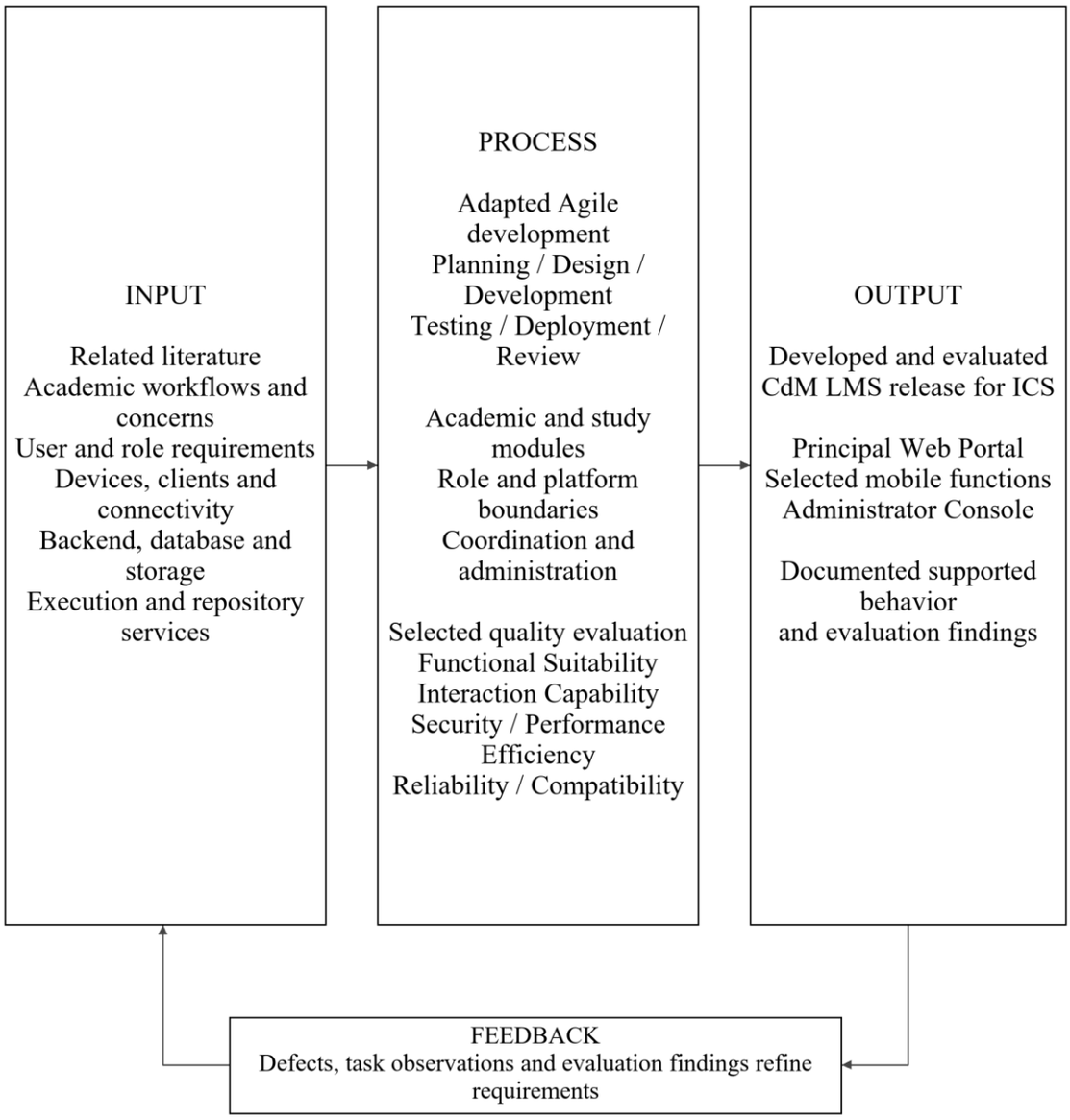


Figure 1

Conceptual framework of CdM LMS using IPO with feedback

The conceptual framework follows the Input–Process–Output (IPO) model with feedback, as shown in Figure 1. It organizes the study's requirements and resources, development and evaluation activities, and expected system output. Purwanto et al. (2025) examined limitations of linear IPO representations and proposed an IMOI adaptation in which outputs and feedback inform subsequent iterations. The present study retains IPO while incorporating feedback into requirement refinement and succeeding development cycles.

The Input component comprises related literature, initial academic practices and concerns within ICS, and system requirements. The literature covers academic management, programming coursework, learning support, and collaboration. Local requirements gathering will examine the use of separate academic and productivity tools, resource management, faculty coordination, and access conditions. These concerns will be investigated rather than reported as established local findings in advance.

Technical inputs include development workstations, supported client devices, Next.js services, PostgreSQL, web browsers, the Expo mobile application, and the Electron Administrator Console. Connectivity, persistent storage, programming execution, repository access, authorization, continuity arrangements, and test configurations will define the operating environment. Estimated equipment specifications will be distinguished from configurations demonstrated during evaluation.

The Process component combines iterative development with requirements-based testing and software-quality evaluation. Planning, Design, Development, Testing, Deployment, and Review will organize successive increments. The modules will address academic management, computing coursework, personal study, gamification, collaboration, academic coordination, and administration. Their availability will follow Student, Instructor, Super User, and Administrator permissions and the complementary responsibilities of the three applications.

The evaluation will use six selected ISO/IEC 25010:2023 characteristics: Functional Suitability, Interaction Capability, Security, Performance Efficiency, Reliability, and Compatibility (ISO, 2023). Respondent ratings will describe experiences with assigned tasks, while technical checks will examine calculations, access restrictions, record handling, performance, and shared information. Abid et al. (2024) provide a feature-based example of systematic LMS evaluation; the study's quality characteristics are derived from ISO/IEC 25010:2023.

The Output is the developed and evaluated CdM LMS release for ICS. It will provide a principal Web Portal, selected mobile support, and a native Administrator Console around shared records and authorized workflows. Feedback from testing, evaluation, and review will return to the

Input component to refine requirements and priorities. Revised inputs will guide subsequent development cycles. Findings will concern the evaluated release and will not establish long-term educational effects or institution-wide adoption.

1.5 OPERATIONAL FRAMEWORK

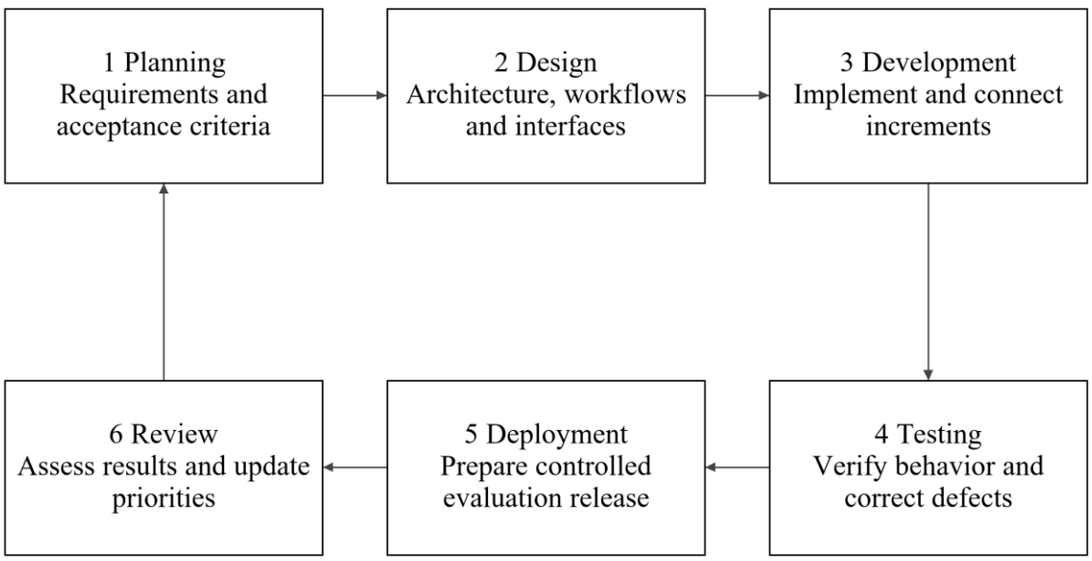


Figure 2

Adapted Agile development cycle for CdM LMS

The researchers will use an adapted Agile development approach consisting of Planning, Design, Development, Testing, Deployment, and Review. Alsaqqa et al. (2020) describe Agile values, iterative methods, and responsiveness to changing requirements. The six stages used here are the project's operational organization rather than a universal Agile sequence. Each cycle will address prioritized requirements and produce an increment for examination and refinement.

Planning Phase. The researchers will gather local requirements from ICS students and instructors. Llamas et al. (2025) identify LMS usability and workflow concerns, while Guico (2026) highlights preparation, resource access, and training needs. These findings inform topics for local investigation. Requirements will specify role responsibilities, platform coverage, expected behavior, and acceptance criteria in a prioritized backlog.

Design Phase. Architecture, database relationships, workflows, and interfaces will be prepared from the requirements. Flowcharts, ERDs, and DFDs will identify information movement and component responsibilities. Attention to navigation and task feedback is supported by the user-experience concerns examined by Laspiñas and Almerez (2024). The design will separate academic supervision from teaching and administration and protect personal academic records.

Development Phase. Selected functions will be implemented incrementally using the Next.js backend, PostgreSQL database, and applicable clients. Development will connect academic management, programming coursework, study tools, gamification, collaboration, coordination, and administration. Each increment will be checked against its acceptance criteria. The release record will distinguish implemented, proposed, tested, and evaluation-enabled functions.

Testing Phase. Controlled accounts and datasets will be used to examine correctness, authorization, integrity, integration, performance, and failure handling. The evaluation release will also undergo role-appropriate task assessment aligned with the six selected ISO/IEC 25010:2023 characteristics (ISO, 2023). Technical results and respondent ratings will be reported separately. Defects will be corrected and retested before affected functions are evaluated.

Deployment Phase. A reviewed release will be configured in a controlled environment. The researchers will verify startup, backend connectivity, accounts, client builds, and available continuity procedures. Participants will receive task guidance. The release version, enabled modules, device configurations, and unresolved limitations will be recorded; controlled evaluation will not be described as institution-wide deployment.

Review Phase. The researchers will examine their technical test records, questionnaire responses, and task feedback from eligible ICS students and instructors. Findings will identify supported behavior, defects, and refinement priorities. These priorities will return to the development backlog, and corrected functions will undergo further testing.

1.6 SCOPE AND LIMITATIONS

The study covers the development and software-quality evaluation of CdM LMS for the Institute of Computing Studies at Colegio de Montalban, including BSIT and BSCpE students, instructors, and supporting academic and administrative workflows. Participation is not restricted to one student year level. IBE and ITE configuration is a technical application capability; those institutes are outside the research population and generalization of findings.

The Web Portal is the principal academic environment and the initial platform for the coordination expansion. Mobile provides selected academic and productivity functions according to the evaluated build. The native Desktop Application is the Administrator Console. Super User oversight is web-based; native mobile parity for the expansion and native Super User console entry are excluded. A role and platform matrix will identify the functions actually tested.

Functional coverage includes course resources and submissions, the Dynamic Gradebook, descriptive analytics, separate CodeLab and Git coursework workflows, Knowledge Exchange, personal notes and study tools, gamified progression, faculty organization, restricted supervision, schedules, targeted events, and administration. Dean and Program Head are academic designations attached to eligible accounts. They do not replace the four primary roles or automatically grant Super User privileges.

Evaluation respondents will consist only of eligible ICS students and instructors with prior LMS experience, selected through criterion-based purposive sampling. Participants must voluntarily agree and perform relevant tasks before rating them. The researchers will conduct technical checks using controlled accounts and datasets. Administrator and Super User functions remain within system development and technical verification, but they will not form separate respondent groups. Instrument review by the adviser and faculty validators is distinct from participation in the evaluation sample.

Operation depends on supported devices, connectivity, and service availability. The study does not claim universal operating-system support, identical platform functions, uninterrupted service, or full offline coursework. Hardware figures are estimates until tested. Programming

languages, repository access, reporting, and continuity coverage will be limited to the documented evaluation release.

Academic analytics describe existing records. Rule-based attention indicators do not predict future achievement. CodeLab checks predefined cases, while Git history supports review without proving authorship or eliminating plagiarism. Tab switching, copy/paste, and fullscreen events are contextual assessment indicators rather than conclusive cheating evidence. Optional grade incentives will be evaluated only when their effect and institutional authorization are documented; EXP, forum reputation, and academic grades remain distinct measures.

Scheduling uses Asia/Manila, selected weekdays, one shared time pair, date bounds, and effective-dated revisions. Individual future classes may be cancelled, rescheduled, or restored. Arbitrary recurrence, multiple weekly time patterns, automatic attendance, holiday cancellation, room-conflict prevention, and timetable optimization are excluded. Student recurring timetable projection is excluded; authorized events and coursework deadlines remain available.

Supervision excludes student identities, individual grades, submissions, attempts, answers, and teaching controls. Event audiences govern LMS visibility without implying external delivery or Google synchronization. Note–event conversion creates private snapshots; later edits do not propagate. The descriptive study will not determine long-term changes in achievement, motivation, employability, or study habits. Findings apply to the selected ICS participants, tasks, and evaluated release.

1.7 SIGNIFICANCE OF THE STUDY

The proposed CdM LMS may contribute to the Institute of Computing Studies (ICS) by providing an integrated environment for managing academic and technical learning activities across Web, Mobile, and Desktop platforms. The study may provide value to the following:

Institute of Computing Studies and Colegio de Montalban. The study may provide an institutional basis for organizing related academic information and examining whether a tailored

LMS meets the needs of ICS. Its documented requirements and evaluation findings may support decisions about further development, maintenance, and possible adoption.

Students. The system is intended to provide a coordinated place for accessing coursework and feedback and managing personal study activities. Students may benefit from clearer access to their own academic information and opportunities for moderated peer support. These expected benefits remain distinct from measured improvements in academic achievement.

Instructors. The study may support more consistent handling of course resources, submissions, grading records, programming activities, and academic communication. Evaluation findings may identify interaction or workflow difficulties that require correction before broader use.

Super Users and Program Heads. Restricted Super User supervision may support authorized academic oversight. Program Heads may benefit from faculty information and designation-specific event publication. These responsibilities will remain distinct from teaching and system administration.

Administrators. The dedicated console is intended to support controlled account operations, system records, reporting, and supported continuity functions. Documenting access boundaries and audit behavior may assist in maintaining accountability for administrative actions.

Future researchers and developers. The study may provide a reference for examining an institutional LMS with complementary applications, computing-specific workflows, and limited academic supervision. Its limitations and evaluation findings may also identify questions requiring separate research, including longer-term educational outcomes and broader platform coverage.

1.8 GENERAL FEATURES OF THE SYSTEM / STUDY

CdM LMS comprises complementary applications connected through shared backend services and a centralized database. The following groups describe intended feature coverage. A release record will distinguish existing modules, planned extensions, tested functions, and functions enabled for evaluation. Availability is governed by role and platform coverage.

Core Functional Features

Academic and Course Management

Authorized Instructors organize courses, sections, learning materials, annotations, assignments, announcements, and deadlines. Students request enrollment using valid course codes; access is granted after approval. Streams, classwork, people, and grades organize approved course information and permitted records.

Grading and Academic Analytics

The Dynamic Gradebook supports configurable assessment components and weights, score recording, computed grades, authorized personal grade views, descriptive charts, and permitted CSV exports. Rule-based attention indicators summarize recorded conditions; they do not predict achievement.

Programming and Collaborative Learning

CodeLab supports browser-based code execution and assessment against predefined test cases. Git-integrated coursework supports a separate repository submission and Instructor review workflow. Knowledge Exchange supports questions, answers, resources, authorized moderation, Faculty-Verified Answers, and contribution reputation.

Personal Study and Gamification

Personal notes, tasks, flashcards, review and mastery records, focus timers, and study-session records support organization and practice. Configured EXP, levels, achievements, badges, streaks, and leaderboards describe eligible participation. EXP, academic grades, and forum reputation remain distinct.

Academic Coordination and Calendar

The expansion will provide a faculty directory and organizational chart, a restricted academic operations hub, safe course supervision, effective-dated weekly schedule revisions, future occurrence exceptions, audience-targeted events, and private note–event snapshots. Day, Week, Month, and Year

views will combine permitted saved events, applicable generated classes, and coursework deadlines. Student recurring timetable projection is outside the stated release.

User Features

Role-Based Workspaces

Students access approved coursework, personal grades, study tools, and permitted collaboration. Instructors manage assigned teaching activities and related coordination. Super Users perform restricted web-based academic oversight without grading or system administration. Administrators use the native Console for protected account, configuration, audit, reporting, and supported continuity operations. Dean and Program Head are academic designations; only authorized Administrators provision or revoke Super User accounts.

System Capabilities

Shared Information and Automated Processing

Role-appropriate dashboards present relevant course, progress, task, coordination, or system information. Shared records are retrieved through authorized backend workflows. The system computes configured grades and progression values and produces permitted calendar outputs. In-LMS updates and available platform-help functions follow the evaluated release; targeted event visibility does not imply external-calendar synchronization or push delivery.

Technology Integration

Connected Applications and Data Services

The Next.js Web Portal and backend connect PostgreSQL records, the Expo/React Native mobile client, and the Electron Administrator Console. Programming execution, repository integration, file storage, and supported account-related email use their configured services. Final versions, supported languages, client builds, and enabled services will be recorded in Chapter 3.

Security Features

Account and Information Protection

Credential authentication, account-status checks, and server-side authorization restrict access by role, institute, ownership, enrollment, and related resources. Any verification or approval requirements must agree with the evaluated workflow. Restricted supervision excludes student identities, grades, submissions, attempts, and answers. Designated changes are audited; constraints, transactions, and reviewed update rules protect record integrity. Repository authorization remains separate from LMS login.

Unique or Innovative Features

ICS Workflow Integration

The project brings academic management, separate execution and repository coursework, personal study, participation records, and moderated collaboration into an ICS-specific environment. Complementary platform responsibilities and bounded academic oversight define its intended configuration. The proposed calendar expansion connects faculty coordination and private organization while preserving schedule history and information boundaries. These contributions will be evaluated within ICS without claiming that comparable functions are absent from other LMS products.

Table 1. Role and platform responsibilities

Role	Principal interface	Permitted responsibility	Boundary
Student	Web; selected mobile functions	Approved coursework, own grades, study tools and collaboration	No faculty directory or supervision
Instructor	Web; selected mobile functions	Assigned teaching, grading, enrollment and schedules	Program Head capabilities require designation
Super User	Web academic operations hub	Safe supervision and narrow faculty coordination	No grading, assessments, schedule editing or system administration
Administrator	Native Desktop Console	Protected account/configuration, audit and supported continuity operations	Additional browser/mobile administration is outside this evaluation scope

1.9 DEFINITION OF TERMS

Academic Calendar - The interface presenting authorized saved events, generated class occurrences, and coursework deadlines through Day, Week, Month, and Year views.

Academic Operations Hub - The web interface through which a Super User views authorized academic summaries, faculty information, timetable filters, and course-supervision links.

Administrator - The system role responsible for authorized account, permission, configuration, log, reporting, and continuity operations through the native Administrator Console.

Agile Development - The iterative development approach in which requirements are implemented, tested, reviewed, and refined through successive project cycles.

Audit Log - A record of designated actions and changes used to support accountability, including authorized administrative and academic-leadership operations.

BSCpE - Bachelor of Science in Computer Engineering, one of the ICS programs within the study context.

BSIT - Bachelor of Science in Information Technology, one of the ICS programs within the study context.

CdM LMS - The proposed interconnected learning management environment developed for the Institute of Computing Studies at Colegio de Montalban.

Centralized Database - The shared PostgreSQL data store supporting authorized academic and system records used by the applicable applications.

CodeLab - The in-browser programming environment for writing, executing, and submitting code assessed against predefined test cases.

Course Supervision - Restricted, read-only access to course metadata, instructor attribution, timetable, announcements, syllabus, and learning-material summaries.

Dean - An academic leadership designation represented in the institute directory; the designation itself does not grant additional system permissions.

Dynamic Gradebook - The grading module that uses configured components, weights, and recorded scores to compute and display authorized grade information.

Effective Date - The date from which a particular timetable revision governs the generation of applicable recurring class occurrences.

Experience Points or EXP - Configured participation points used for gamified progression and related rewards; they are not equivalent to course grades or forum reputation.

Faculty Directory - The authorized listing of ICS faculty, program affiliations, leadership designations, and relevant assigned-course summaries.

Faculty-Verified Answer - A Knowledge Exchange response marked by authorized faculty through the platform's answer-verification process.

Flashcard Review - Repeated study of saved question-and-answer cards using the review and mastery functions available in the evaluated release.

Gamification - The use of configured points, levels, badges, achievements, streaks, and leaderboards to support participation in permitted LMS activities.

Git-Integrated Submission Engine - The workflow for repository-based coursework submission and authorized instructor review of applicable programming activities.

Institute of Computing Studies or ICS - The institute at Colegio de Montalban that defines the development context and research-evaluation scope of the study.

Instructor - The system role responsible for authorized teaching, assessment, grading, and course-related communication for assigned courses.

Knowledge Exchange - The moderated environment for academic and technical questions, answers, and resource sharing, with faculty verification and reputation mechanisms.

Learning Management System or LMS - The application environment used in this study to organize course resources, coursework, assessment, academic records, and related support workflows.

Program Head - An academic designation attached to an eligible ICS Instructor account for a specified program, with defined coordination privileges.

Reputation Points - Configured points associated with Knowledge Exchange contributions, maintained distinctly from EXP unless an explicit rule connects them.

Role-Based Access Control or RBAC - The mechanism that restricts system capabilities according to assigned roles, supplemented by institute, ownership, enrollment, and related-resource checks.

Schedule Exception - A record modifying an individual class occurrence through cancellation or rescheduling, or supporting restoration of its original occurrence.

Snapshot Conversion - Creation of a private note or calendar event from accessible source content at the time of conversion, without continuous synchronization.

Student - The system role permitted to access enrolled-course content, submit assigned work, view authorized personal academic records, and use available learning-support functions.

Super User - The system role for limited academic oversight and coordination, separate from Instructor and Administrator privileges.

Targeted Event - A saved academic event whose visibility and management are governed by its designated personal, course, institute-faculty, or program-instructor audience.

Timetable Revision - An effective-dated version of a course schedule used to preserve the historical representation of recurring class times.

Visual Grade Analytics - Descriptive charts and summaries derived from available grade and performance records, without predictive or AI-based inference.

CHAPTER 2: REVIEW OF RELATED LITERATURE

Digital technology has changed how educational institutions organize instruction and manage academic information. Learning Management Systems (LMS) provide a shared environment for distributing course materials, managing assignments, recording grades, and communicating with students. Access through different devices also creates opportunities to connect classroom activities with independent study. These functions provide the foundation for examining how an LMS can support academic work within higher education.

In the Philippine context, the studies reviewed in this chapter examine LMS adoption, user experience, accessibility, and institutional support. They also explore programming practice, gamification, personal learning strategies, and online collaboration. Their findings provide different perspectives on digital learning, including the difficulties users encounter and the conditions that support meaningful participation. Understanding these findings helps guide the selection and integration of functions appropriate to the Institute of Computing Studies (ICS) at Colegio de Montalban.

The project developers reviewed scholarly articles, conference papers, and system-based studies to establish the foundation for the proposed Learning Management System. Philippine and international studies are organized under eight topic groups: Multi-Platform Learning Management Systems; Course Management and Academic Communication; Programming Practice and Automated Assessment; Git-Based Coursework Submission; Gamification and Student Engagement; Personal Study Tools and Self-Regulated Learning; Online Collaboration and Knowledge Sharing; and Academic Coordination and Information Boundaries. The discussion compares their findings, explains their connection to the proposed system, and identifies the research gap that the study will address.

2.1 Related Literature

2.1.1 Multi-Platform Learning Management Systems

Access through different devices should support useful academic tasks and an understandable learning environment. Alfalah (2023) surveyed 258 students from Hail University, Aljouf University, and Northern Border University in Saudi Arabia about mobile LMS adoption, including the Blackboard application. Performance expectancy, effort expectancy, and learning intention were associated with behavioral intention to use the system. In the Philippines, Dela Rosa (2023) examined Canvas Free for Teachers with 132 third-year BSIT students taking Web Systems and Technologies 2 at Bulacan State University. Students evaluated its lesson modules and assessments favorably. Both studies connect adoption with users' experiences of academic functions, although Alfalah examined mobile-use intentions and Dela Rosa examined acceptance within a specific programming course.

Philippine development studies provide more direct examples of designing learning access across devices. Quimod et al. (2023) developed e-Modyul, a mobile and web LMS for Technical-Vocational-Livelihood education, and evaluated it with 10 teachers and 20 students. Its approach supports accessing lessons and assessments through different operating systems. Ramos et al. (2024), researchers associated with the University of the Philippines Open University (UPOU), developed a Filipinized LMS responsive to Filipino nursing educators and learners. Their design considered connectivity limitations, multimodal learning materials, communication, and collaboration. These studies share an emphasis on access, but e-Modyul focuses on device coverage, while the Filipinized LMS emphasizes adaptation to its users and educational context.

Platform access also depends on the quality of everyday workflows. Llamas et al. (2025) examined the Moodle-based LMS at Mindanao State University–Iligan Institute of Technology (MSU–IIT), involving six academics and six fourth-year students. They identified concerns in usability, communication, course building, monitoring and assessment, content management, and technical support. Their findings qualify the favorable acceptance evidence from Dela Rosa: providing an LMS does not remove all difficulties in using it.

Taken together, the reviewed studies support designing complementary interfaces around the tasks and access conditions of the intended users. For CdM LMS, the web application will provide the principal academic environment, the mobile application will support selected student and instructor functions, and the desktop application will serve the Administrator. Multi-platform access therefore concerns coordinated responsibilities and consistent shared records rather than identical functionality on every device. The reviewed studies inform this arrangement, while its suitability for ICS will require evaluation of the implemented applications.

2.1.2 Course Management and Academic Communication

Course management connects learning materials, activities, academic records, and communication within a clear workflow. Peralta (2024) developed a course management system for Centro Escolar University and evaluated it with 100 students and eight faculty members. It organized materials, announcements, assignments, quizzes, and grades and included Short Messaging Service notifications. Llamas et al. (2025), however, identified course-building, content-management, communication, and assessment concerns in the MSU–IIT LMS. These studies suggest that organizing functions within one platform is useful, but users must still be able to complete their tasks without unnecessary difficulty.

The communication experience also varies across educational settings. Acala and Dimasacat (2023) examined perceptions of 74 private senior high school students in Manila and reported favorable views of LMS-supported interaction and learning. They recommended training to help students and teachers use the available functions. Laspiñas and Almerez (2024), in contrast, surveyed 33 business graduate students at an unnamed university in Cebu City and reported generally neutral evaluations of the LMS's pragmatic and hedonic qualities. Their findings address different populations, yet both indicate that communication and task support should be assessed through users' experiences rather than inferred from the presence of features.

Institutional support affects whether these workflows can be maintained. Guico (2026) examined 182 faculty members and 29 administrators from 11 higher education institutions in

Batangas, including Batangas State University, Lipa City College, and Rizal College of Taal. Reported concerns included preparation time, resistance to change, access to information and communication technology resources, and continuing training. These findings place course management within faculty support and institutional coordination.

Across the reviewed studies, effective academic communication depends on understandable course organization, appropriate user preparation, and the support needed to sustain the system. This evidence informs CdM's course materials, announcements, submissions, feedback, dynamic grading, and descriptive grade analytics. It also supports considering faculty organization, schedules, and targeted academic events as part of course coordination. Peralta's SMS function remains a comparison feature of that system; the proposed CdM workflow uses its own documented notification functions. Requirements gathering at ICS will establish the local difficulties to be addressed.

2.1.3 Programming Practice and Automated Assessment

Programming practice requires suitable exercises and feedback that explains performance against clear criteria. Messer et al. (2024) reviewed 121 papers on automated grading and feedback tools for programming education. Many tools evaluated correctness through dynamic methods such as unit tests, while readability, maintainability, and documentation received less attention. Novak and Kermek (2024), from the University of Zagreb in Croatia, examined automation for complex programming assignments using unit testing and custom scripts. Both studies support automating selected checks while retaining instructor judgment for aspects that those checks cannot adequately assess.

Philippine research adds considerations about introductory practice and access. Asuncion et al. (2022), affiliated with Ateneo de Manila University and the National Olympiad in Informatics–Philippines, presented the ABC Workbook and AutoJudge++. The project combines introductory Python exercises with an offline desktop checker that compares outputs against test cases. Classroom effectiveness evaluation remained future work. Arada (2025) examined Tinkercad with 42 first-year Computer Technology students at a Philippine state university. The platform received a below-

average System Usability Scale score of 58, despite many students expressing willingness to use it. These studies concern different tools, but together they show that opportunities to practice should be considered alongside exercise design, usability, and access conditions.

Assessment can also concern development processes beyond the output of an individual program. Fernandez-Gauna et al. (2023), from the University of the Basque Country in Spain, described automated feedback and assessment for team-coding assignments in a DevOps context. Their approach used team and individual indicators and periodic reports to support monitoring. Compared with the test-case approaches examined by Messer et al. and Asuncion et al., this work illustrates a broader form of assessment involving collaborative development activity. It does not establish that development indicators can replace assessment of the submitted program.

The reviewed studies support matching automated feedback to the assignment and preserving the limits of what is measured. CdM's CodeLab will evaluate programming work against predefined test cases and provide understandable results. Its checks will concern documented expected behavior rather than complete program quality. Git-based coursework review will remain a separate workflow. The related studies therefore guide test design, feedback presentation, and usability without expanding CodeLab into automated code repair, artificial intelligence grading, or DevOps-based team assessment.

2.1.4 Git-Based Coursework Submission

Repository-based coursework can connect code submissions with an organized record of development and instructor review. Schauer et al. (2024), from Heriot-Watt University in the United Kingdom, developed a framework integrating Canvas LMS and GitLab. They identified fragmented information and poor user experiences when the platforms were used independently and implemented eight applications to connect their workflows. Orvalho et al. (2024), affiliated with the University of Lisbon and Czech Technical University in Prague, presented GitSEED, a GitLab-backed automated assessment tool. Both approaches connect academic work with version control, although Schauer et

al. emphasize LMS integration and GitSEED provides assessment and feedback through repository workflows.

Evidence about students' use of repositories complements these system-development approaches. Patani et al. (2024) examined GitHub use in a software engineering course with 319 undergraduate students, using surveys and repository records such as commits, pull requests, and issues. They reported improvements in collaborative development and teamwork after the course experience. Fernandez-Gauna et al. (2023) used repository-related development indicators to provide feedback on team-coding assignments. Together, these studies show how repositories can make development activity available for educational review, but their measures extend beyond recording a final coursework submission.

The reviewed studies support examining how academic instructions, repository records, and instructor feedback can be connected. For CdM LMS, Git authorization will support the documented repository submission and review process, while LMS account access will follow its separate authentication and approval rules. Repository submissions will not automatically become CodeLab assessments or commit-based academic grades. The project's contribution is to evaluate its particular coursework workflow for ICS, informed by the integration concerns identified by Schauer et al. and the educational repository use described in the other studies.

2.1.5 Gamification and Student Engagement

Gamification can encourage participation when rewards are connected to meaningful learning activities. Ahmed et al. (2025) evaluated a gamified, mobile, cloud-based LMS with 60 second-year students taking Object-Oriented Programming at Mansoura University in Egypt. The experimental group obtained higher post-test achievement and engagement scores than the comparison group. Jaramillo-Mediavilla et al. (2024) synthesized gamification research and reported favorable findings in parts of the literature while identifying gaps in examining motivation and academic performance together. These studies support investigating gamification, but the outcomes of a particular intervention should not be treated as a guaranteed effect of every reward system.

The design of participation incentives matters. Jack (2025), in a conference abstract, discussed points and leaderboards in a flipped statistics classroom and considered benefits for motivation and engagement alongside the risks of superficial engagement and demotivation. Philippine evidence from Fojas (2022) examined gamification in Moodle courses at UPOU, comparing a gamified cohort with earlier non-gamified cohorts and collecting student feedback. The reported differences favored the gamified course, but forum-grading arrangements also differed between cohorts, and some students described anxiety or limited interest in rewards. Both studies show why participation counts and positive feedback need to be interpreted in relation to the design of the activities.

Benben and Bug-os (2022) studied Quizizz-based formative assessment with Grade 10 physics students at Central Mindanao University Laboratory High School in Bukidnon. Achievement and motivation increased after the intervention, although the reported learning gain was low. Its one-group pretest-posttest design and secondary-school setting differ from Ahmed et al.'s computing-course comparison. The studies therefore provide complementary evidence about possible motivational benefits while leaving questions about transfer to other subjects and populations.

Across the reviewed studies, gamification is most relevant when its rules encourage participation without obscuring academic expectations. CdM LMS will use EXP, levels, achievements, streaks, and leaderboards under defined rules. These records will remain separate from academic grades and Knowledge Exchange reputation. The literature supports the rationale for these mechanisms and the need to examine users' experiences, while the proposed descriptive software-quality evaluation will not establish their long-term causal effects on achievement or motivation.

2.1.6 Personal Study Tools and Self-Regulated Learning

Personal study tools can support planning, active review, and reflection when learners use them purposefully. Du et al. (2025) examined an LMS enhanced with self-regulated learning support among 69 management undergraduates taking mathematics at an unnamed large public university in China. Students reported stronger self-regulation, but an examination advantage appeared on the first test and not the second. Heikkinen et al. (2023), in a review of 56 studies, found that only 46 percent

of the reviewed learning-analytics interventions reported positive effects on learning. Both studies indicate that learning support deserves investigation, while access to monitoring tools alone does not ensure improved outcomes.

Students' interactions with LMS tools help explain differences in how learning support is used. Çakiroğlu et al. (2024) examined a third-year Operating Systems and Applications course at an unnamed public university in Turkey. They identified different learner profiles and relationships between assessment interactions and self-regulation skills, including goal setting, self-evaluation, and help-seeking. In the Philippines, Funa et al. (2023) surveyed 301 preservice teachers in the Bicol region and supplemented the survey with interviews. Participants reported relatively strong environment structuring and goal setting but also difficulties with task strategies, time allocation, priorities, and interaction. These studies connect self-regulation with both individual learning practices and the conditions in which academic tasks are completed.

Active review provides another form of personal learning support. Pan et al. (2022) compared user-generated and premade digital flashcards in six experiments involving delayed tests. Creating flashcard content generally supported memory and, in several conditions, application performance. Unlike the broader LMS and analytics studies, this research examines a specific review activity. It offers a direct basis for considering student-created flashcards alongside the organization and reflection functions discussed by Du et al., Çakiroğlu et al., and Funa et al.

Together, the reviewed studies support making planning, review, and progress information available within the learning environment. For CdM LMS, personal notes, tasks, flashcards, review and mastery records, and focus-study records will provide ways to organize and revisit academic work. Descriptive displays can help users inspect their saved records, but their presence will not be treated as evidence of improved self-regulation. The implemented functions will require evaluation for usability and correct behavior, while their educational effects remain a possible direction for later research.

2.1.7 Online Collaboration and Knowledge Sharing

Online knowledge sharing depends on the quality of interaction and the way discussions are organized. Oyarzun and Martin (2023) reviewed 63 studies on online learner collaboration and examined technologies, instructional design, facilitation, and outcomes. Their review connects collaboration with the conditions supporting participation. Bach and Thiel (2024) surveyed 298 undergraduate teacher-education students in asynchronous university courses and examined cognitive, metacognitive, and relational dimensions of interaction. Their findings concern associations with self-reported learning gains and satisfaction rather than causal effects. Both studies indicate that a shared online space should be assessed through the exchanges it supports.

Structured discussions and instructor involvement provide more specific guidance. Mudau and Van den Berg (2023), from the University of South Africa, examined online discussions involving 11 master's students in education. They identified the importance of clear guidelines, structured questions, and opportunities for discussion within a community of inquiry. Almodiel (2022) analyzed knowledge construction in asynchronous discussions within a graduate colloquium at UPOU. The study used the Interaction Analysis Model to examine the exchanges through which learners shared and developed knowledge. These studies support considering discussion structure and facilitation alongside opportunities for students to contribute.

Participation measures should also reflect interaction between users. Kumi (2023) examined discussion-forum exchanges in an undergraduate class with 31 students using social network analysis. The analysis revealed structural and relational aspects of participation that simple counts or self-reports could overlook. This complements Almodiel's examination of knowledge construction: both approaches look beyond the number of messages to consider how students interact. Their analytical methods are research approaches rather than functions automatically required in the proposed CdM system.

The reviewed studies support designing Knowledge Exchange as an organized space for academic questions, answers, and resource sharing. Moderation and Faculty-Verified Answers will

provide project-specific ways to manage contributions and identify instructor-reviewed responses. Reputation will describe forum participation under its configured rules and remain separate from EXP and academic grades. The literature provides reasons to examine participation, facilitation, and contribution quality, while the suitability of CdM's verification and reputation mechanisms will require its own evaluation. The proposed forum supports asynchronous exchanges; the reviewed studies do not require adding live meetings or collaborative document editing.

2.1.8 Academic Coordination and Information Boundaries

Institutional LMS support extends beyond individual coursework. Guico (2026) identifies faculty preparation, resource access, and continuing training as management concerns. Llamas et al. (2025) identify monitoring, assessment, communication, and course organization as user-experience concerns. Together, these findings provide reasons to examine faculty coordination and access to appropriate course information. They do not validate the particular schedule schema or supervision policy proposed for CdM LMS.

The project's academic coordination requirements will therefore be established through local consultation and explicit role policies. Directory information, timetable revisions, event audiences, and restricted supervision will be tested against those requirements. Academic designation and technical privilege will remain separate; shared calendar visibility will not imply access to private notes or individual assessment records.

2.2 Supporting Quality Framework and Local Study Context

ISO/IEC 25010:2023 provides the product-quality model supporting the study's selected evaluation characteristics: functional suitability, interaction capability, security, performance efficiency, reliability, and compatibility (International Organization for Standardization, 2023). The standard supports the quality criteria used in this study. Questionnaire feedback will describe respondents' evaluations, while calculation checks, access tests, and integration verification will provide evidence about implemented behavior.

The reviewed Philippine studies inform the project's computing-course, LMS, programming, and collaborative workflows. Their findings do not establish conditions within ICS. Local requirements gathering will identify the needs and experiences of the intended users.

2.3 Related Systems

The following systems provide documented examples of course organization, progress information, and repository-based workflows. The comparison uses the capabilities described in the cited sources rather than an exhaustive feature audit. It does not assume that these platforms lack other functions or cannot be extended.

2.3.1 Moodle

Moodle's publication on LMS 4.5 described improvements to assignment interaction and course subsections for organizing content (Moodle, 2024). Llamas et al. (2025) examined its institutional use at MSU–IIT, while Fojas (2022) and Almodiel (2022) examined gamification and discussions in the UPOU context. These sources show that Moodle can support several forms of academic work. For CdM LMS, the relevant comparison concerns the organization of coursework and the experience of completing institutional tasks.

2.3.2 Google Classroom

Mennuti (2024) described targeted assignments, rubrics, and analytics for grades, completion rates, and missing work in Google Classroom. The analytics described were associated with specified education editions or upgrades. These functions provide comparison points for assignment management and descriptive progress information. Their documentation does not establish the quality of the proposed CdM implementation or demonstrate that it will outperform Classroom.

2.3.3 Canvas and GitLab Integration

Schauer et al. (2024) implemented a framework connecting Canvas and GitLab to reduce fragmented workflows and support feedback. Its eight applications demonstrate that existing LMS and version-control platforms can be connected for educational use. This is a direct comparison for

the integration concern in CdM's Git submission workflow. The technologies and automation described in that framework remain distinct from CdM's specified implementation.

2.3.4 GitSEED

Orvalho et al. (2024) presented GitSEED as a GitLab-backed automated assessment tool supporting programming and software engineering education. It uses repositories as part of submission and feedback workflows. Its approach provides a comparison for connecting coursework with version control, but its automated assessment capabilities extend beyond CdM's separate repository-submission and instructor-review process.

2.4 Synthesis of the Literature

The literature connects LMS adoption with useful academic functions, understandable interaction, and conditions supporting continued use. Alfalah (2023) and Dela Rosa (2023) address acceptance, while Quimod et al. (2023), Ramos et al. (2024), and Llamas et al. (2025) provide development and user-experience perspectives. Peralta (2024), Guico (2026) further connect coursework with communication and institutional support. These studies support examining connected workflows and appropriate access across CdM's complementary applications.

Programming and repository research address related but distinct academic processes. Messer et al. (2024), Asuncion et al. (2022), and Novak and Kermek (2024) inform checks against specified programming criteria. Schauer et al. (2024), Orvalho et al. (2024), and Patani et al. (2024) inform repository use and integration. The comparison supports keeping CodeLab's test-case assessment separate from Git-based coursework review, with instructors retaining responsibility for judgments outside the automated checks.

Gamification, personal study support, and collaboration concern different aspects of participation. Ahmed et al. (2025), Jack (2025), and Fojas (2022) provide evidence about rewards and engagement. Du et al. (2025), Heikkinen et al. (2023), Funa et al. (2023), and Pan et al. (2022) address regulation and active review. Oyarzun and Martin (2023), Almodiel (2022), and Mudau and

Van den Berg (2023) emphasize interaction and facilitation. Their combined implication is to provide learning support with clear purposes and evaluate how the intended users experience it.

For the present study, these findings inform the development and descriptive evaluation of an LMS configured for ICS. Evidence from other institutions provides design guidance and comparison points. The study will establish its own findings about software quality and workflow suitability, while recognizing that user ratings and functional verification do not establish long-term educational effects.

2.5 Research Gap

The reviewed studies address digital learning and institutional LMS utilization but do not evaluate the proposed CdM LMS configuration for ICS at Colegio de Montalban. The research gap is contextual and evaluative: evidence is needed about how its combined workflows, roles, and complementary applications operate in the intended setting. One part of the gap concerns functional integration. CdM LMS will combine routine academic management with separate CodeLab and Git-submission workflows, learning-support tools, and moderated collaboration. Its evaluation must determine whether the selected functions produce the expected results and interact consistently within a shared information environment. This is a system-specific question; it is not a claim that no existing platform can provide similar integration.

A second part concerns academic coordination and information boundaries. The proposed expansion introduces an academic Super User, restricted course supervision, faculty organization, effective-dated timetable revisions, class exceptions, and audience-targeted events. The reviewed literature provides organizational reasons to consider coordination, but it does not establish whether CdM's particular implementation preserves schedule behavior, correct audiences, and restricted information access. These properties require requirements-based testing and evaluation of the implemented release.

A third part concerns role-appropriate access across applications. The web environment, complementary mobile client, and native Administrator Console have distinct responsibilities. The

study will examine consistency in their applicable shared records and the quality of the tasks each interface supports, without assuming universal feature parity. Findings will identify the suitability and limitations of this configuration for ICS. The study will address these gaps through development, documented verification, and descriptive software-quality evaluation, producing evidence about the suitability and limitations of the ICS implementation.

CHAPTER 3: METHODOLOGY

3.1 RESEARCH DESIGN

The study will combine iterative system development, requirements-based technical verification, and quantitative descriptive evaluation. Descriptive research systematically characterizes observed conditions and experiences without establishing causal effects (McCombes, 2023). Questionnaires will describe users' experiences with assigned tasks; controlled technical checks will examine correctness, access restrictions, record handling, performance, and integration. The two evidence types will be reported separately and considered together.

The Agile operational framework in Section 1.5 will guide development. Chapter 3 specifies how local requirements, evaluation participants, instruments, tests, and analysis will be organized. The evaluation will address six selected ISO/IEC 25010:2023 characteristics rather than claim assessment of the complete standard or ISO certification.

3.2 SYSTEM AND PROJECT OVERVIEW

CdM LMS connects the principal Web Portal, complementary Expo/React Native application, and native Electron Administrator Console to Next.js backend workflows and PostgreSQL records. Students access approved coursework and their own academic and study information. Instructors manage assigned teaching activities. Super Users perform restricted academic coordination through the web; Administrators operate the native console. Program Head and Dean designations remain separate from those account roles.

The evaluation configuration will identify enabled modules, client builds, roles, and datasets. Existing functions and proposed coordination extensions will have separate implementation and test-status entries. Only demonstrated functions will be included in respondent tasks and reported as evaluated.

3.3 RESPONDENTS

The respondents will consist only of selected ICS students and instructors at Colegio de Montalban. Student respondents must be officially enrolled in the BSIT or BSCpE program, have previous experience using at least one LMS, voluntarily agree to participate, and complete the assigned evaluation tasks. Participation will not be restricted to a single year level. Their tasks will cover approved coursework, learning resources, submissions, personal grades, study tools, and Knowledge Exchange functions available in the evaluated release.

Instructor respondents must currently teach BSIT or BSCpE courses within ICS, have prior LMS experience relevant to their academic responsibilities, voluntarily participate, and complete the assigned tasks. They will evaluate course management, assessment, submissions, grading, enrollment decisions, communication, and applicable calendar functions. Instructors holding a Program Head designation may evaluate its permitted coordination functions. That designation does not grant Super User or Administrator authority.

The capstone adviser and the three proposed faculty validators will review questionnaire content separately from the respondent sample. Their review will not automatically make them respondents; they may participate only if they independently satisfy the Instructor criteria. The researchers will conduct technical verification of restricted supervision and native console functions using controlled test accounts. There will be no IT-expert respondent group or separate expert questionnaire.

Target numbers for Student and Instructor respondents will be finalized after confirming the eligible population and feasible task sessions with the adviser. The researchers will report the invited, participating, and valid responding counts separately for the two groups. No target or completed sample count is asserted before that decision and data collection.

3.4 SAMPLING TECHNIQUE

The study will use criterion-based purposive sampling, a non-probability method in which participants are selected according to requirements relevant to the study. Memon et al. (2025)

emphasize explicit, justified criteria for quantitative purposive sampling. This method is appropriate because Students and Instructors must directly use the functions they will assess. Enrollment or employment at the institution alone will not qualify a participant to rate every platform or system function.

The researchers will screen the Student and Instructor criteria specified in Section 3.3, obtain voluntary consent, and arrange orientation and task sessions through an approved recruitment process. The invitation will explain the purpose, activities, participation conditions, and data use. Sample justification will consider coverage of the two programs, eligible year levels, teaching responsibilities, enabled functions, and feasible evaluation arrangements. Participation and nonresponse will be recorded by group.

Students will assess permitted academic and study functions, while Instructors will assess teaching and course-management workflows. Participants will not receive elevated roles merely to complete the questionnaire. Findings will describe the selected respondents and evaluated release within ICS; purposive selection will not be presented as a probability sample representing the entire student or instructor population.

3.5 INSTRUMENTS AND METHODS FOR DATA COLLECTION

3.5.1 Questionnaires and Instrument Review

Survey questionnaires will be the primary instruments for collecting quantitative user feedback. Two role-specific forms will be prepared: a Student questionnaire and an Instructor questionnaire. Both will align with the six selected ISO/IEC 25010:2023 characteristics: functional suitability, interaction capability, security, performance efficiency, reliability, and compatibility. Items will describe one observable behavior and identify the relevant task. Students and Instructors will rate only functions they have directly used; restricted console or supervision operations will not appear as ordinary user-survey tasks.

Both forms will use Strongly Agree (4), Agree (3), Disagree (2), and Strongly Disagree (1). A separate “Not applicable or not observed” option will receive no numerical score. Security items for respondents will concern observable access boundaries and protection of their own records; resource utilization, prohibited requests, and other internal properties will be examined through the researchers’ technical tests. The survey will not be represented as proof of all technical subcharacteristics.

Before administration, the capstone adviser and three proposed faculty validators from Colegio de Montalban will review clarity, relevance, alignment with the research questions, and suitability for each role. The researchers will revise the instruments and conduct a pilot with eligible Students and Instructors outside the main sample. Reviewer approval will support content review; any internal-consistency assessment will use actual pilot responses and an appropriate analysis rather than infer reliability from approval alone.

Participants will receive orientation and consent information, perform the documented task sequence using authorized accounts, and then complete the applicable questionnaire. The researchers will record respondent group, platform and client build, task completion, and encountered difficulties. Local requirements consultations will precede final development and will remain separate from the post-development evaluation survey.

Table 2. Role appropriate respondent tasks

Respondent	Platform	Representative tasks
ICS Students	Web and selected mobile functions	Approved coursework, own grades, notes/tasks, study tools and Knowledge Exchange
ICS Instructors	Web and selected mobile functions	Assigned materials, submissions, grading, enrollment decisions and schedules
Instructors designated as Program Heads	Web; permitted designation functions	Authorized faculty or program event publication; part of the Instructor group

3.5.2 Local Requirements Gathering

Before finalizing the development backlog, the researchers will seek institutional permission to gather local requirements from ICS students and instructors through guided interviews or small-

group consultations and available workflow descriptions. Topics will cover resource distribution, enrollment, submissions, grading, programming coursework, personal organization, faculty coordination, and device or network access.

With consent, the researchers will record de-identified notes, summarize recurring needs, and validate the resulting requirement statements with appropriate stakeholders. Each accepted requirement will identify its user, applicable platform, expected behavior, access restrictions, and acceptance criteria. These pre-development consultations are distinct from the post-development software-quality questionnaires.

3.5.3 Technical Testing and Observation

A test matrix will map requirements to selected quality characteristics, account roles, platforms, datasets, and expected outcomes. Each case will record the application revision, enabled feature, setup, action, result, error evidence, and retest status. Calculation checks will compare outputs with manually verified expected records. Security cases will invoke prohibited reads/writes directly rather than rely on hidden interface buttons.

The researchers will define workload and dataset sizes before performance testing. Logged-in sessions, request rates, simultaneous programming executions, and upload sizes will be recorded separately; a registered-account count will not establish concurrent capacity. Repeated timing checks will distinguish ordinary retrieval, grade computation, calendar queries, file transfer, and CodeLab execution. Thresholds will be approved and recorded before judging performance.

Reliability checks will examine saved records after refresh, interrupted requests, retries, duplicate conversion, stale saves, and relevant concurrent updates. Researcher-conducted compatibility checks will verify shared records through permitted clients and observe the Administrator Console alongside selected workstation software. Migration and restoration rehearsals will use an isolated database. Unexecuted or failed cases will remain visible in the evaluation record; fresh evidence will support any reported pass.

Table 3. Evidence for selected quality characteristics

Characteristic	Researcher checks	Respondent evidence
Functional Suitability	Expected/actual computations, submissions, schedules and conversions	Ratings of directly used functions
Interaction Capability	Task completion, navigation and feedback observations	Student/Instructor task ratings
Security	Prohibited requests and private-record isolation	Observable permissions and own-record access
Performance Efficiency	Recorded workloads, timings and resource observations	Perceived responsiveness on performed tasks
Reliability	Interrupt/retry, duplicate or stale saves, persistence	Encountered failures during the session
Compatibility	Permitted shared records and native-console co-existence	Client tasks within the respondent's access

3.5.4 Data Analysis and Statistical Treatment

Questionnaire responses will be tabulated by respondent group, item, and quality characteristic. Frequencies and percentages will describe response distributions. The valid count for each item will exclude unanswered and Not applicable or not observed responses; those exclusions will also be reported. N/A will not be treated as zero, disagreement, or a passing result.

For comparability with the coded scale, an item mean will be calculated as the sum of valid response scores divided by the valid response count. Equivalently, the frequency-weighted mean is (4f4 + 3f3 + 2f2 + f1) / (f4 + f3 + f2 + f1). Frequency weighting represents counts of responses, not different importance weights for users or quality characteristics. Response distributions and valid counts will accompany means; median summaries may be reported to retain visibility of the ordered categories.

Characteristic summaries will first be computed for each respondent from the applicable items, then summarized within the Student or Instructor group. The number of applicable items and contributing respondents will be reported. Results from different roles or questionnaire versions will not be pooled into one score unless item equivalence and weighting are explicitly justified. Numeric results will be reported on the 1–4 scale without unapproved labels such as highly acceptable.

Technical results will be reported by test case and characteristic. Correctness and access cases will identify expected/actual outcomes and pass, fail, or not executed status. Timings will identify workload, repetitions, median and observed slow cases; resource observations will identify the measurement conditions. Favorable questionnaire means will not override failed authorization, calculation, or persistence tests. The study will use descriptive summaries and will not claim causal educational effects or population-wide statistical representation.

3.5.5 Participant Protection and Information Handling

The researchers will obtain applicable institutional permission before recruitment or access to restricted records. Participants will receive information about purpose, tasks, duration, data use, and voluntary participation. Declining or withdrawing will not affect grades, employment, or access to ordinary institutional services. Consent will be obtained before task participation and questionnaire collection.

Evaluation identifiers will replace participant names in analysis files. The researchers will use controlled accounts and synthetic or appropriately de-identified academic records for restricted tests. Only authorized members of the research team and approved reviewers will access identifiable materials. Storage location, access controls, retention period, and deletion arrangements will be specified in the approved protocol. Public reports will present aggregate results and avoid exposing credentials, repository tokens, or personal academic records.

3.5.6 Evaluation Release Record

Before recruitment, the researchers will record the application revision, client builds, schema and migration state, enabled modules and languages, repository provider, file limits, device/browser matrix, network conditions, role accounts, datasets, and known limitations. Existing modules, planned extensions, tested functions, and functions enabled for respondent tasks will be identified separately. Administrator and Super User functions will be verified by the researchers with controlled accounts; ordinary respondents will not rate functions outside their authority. Secrets and identifiable academic records will be excluded from manuscript examples.

3.6 HARDWARE REQUIREMENTS

Equipment specifications below are planning estimates for development and a small evaluation pilot. They are not experimentally established minimums or verified concurrent capacity. The actual device and hosting configuration will be recorded before task sessions.

Table 4. Estimated development and client hardware

Equipment	Estimated baseline	Recommended	Purpose
Development workstation	4 cores; 16 GB RAM; 50 GB free SSD	6+ cores; 32 GB RAM; 100 GB free SSD	Build clients and run local checks
Web computer	Dual core; 4 GB RAM; 1366×768; 1 GB free	4 cores; 8 GB RAM; 1080p; 5 GB free	Academic and coordination tasks
Android device	Build-compatible; about 4 GB RAM; 1 GB free	6+ GB RAM; 2 GB free	Enabled mobile tasks
iOS device if evaluated	Compatible iPhone/iPad; 1 GB free	Supported device; 2 GB free	Verified iOS build only
Administrator workstation	64-bit dual core; 4 GB RAM; 2 GB free	4 cores; 8 GB RAM; 5 GB free	Native Console and exports
Network equipment	Reliable router/AP and connectivity	Wired laboratory links and sufficient Wi-Fi capacity	Connect authorized clients/services

Storage allowances cover dependencies, temporary files, cache, and downloads rather than measured installer sizes. Emulators, containers, and datasets may need more development space. Enabled 3D activities require a tested WebGL 2-capable browser/graphics configuration; dedicated graphics is not assumed for ordinary LMS tasks.

Table 5. Proposed pilot hosting and network configuration

Component	Initial estimate	Condition to verify
Application service	2 vCPU / 4 GB RAM on a conventional server	For serverless hosting, record plan/function limits instead
PostgreSQL	2 vCPU / 4 GB RAM; 10–20 GB initial records/index storage	Connection limits, growth and query behavior
Persistent file storage	20–50 GB initial object-storage allowance	Materials, submissions, snapshots and retention
Code execution if self-hosted	Separate 4 vCPU / 8 GB RAM; 30–50 GB storage	Languages, isolation, test limits and execution concurrency
Backups	Separate protected storage	Retention capacity and demonstrated restoration
Individual connection	About 5 Mbps down / 2 Mbps up; prefer 10/5	Measured latency, contention and file sizes

Component	Initial estimate	Condition to verify
Shared lab connection	About 100 Mbps down / 20–50 Mbps up for an initial 20–30-device session	Actual simultaneous workload and connection stability

The initial sizing assumption is approximately 25–50 simultaneously active ordinary-task users, not a research sample size or proven capacity. Code executions will be limited and measured separately. Lower bandwidth may support basic retrieval, while uploads and concurrent activity may need more. Managed-service quotas and the demonstrated workload take precedence over these estimates.

3.7 SOFTWARE REQUIREMENTS

The software record identifies the current development stack and proposed evaluation targets. It will be frozen against the actual release revision; framework support alone does not establish CdM client compatibility.

Table 6. Software environment and release configuration

Software/service	Recorded choice	Purpose/qualification
Development OS	Supported Windows/macOS/Linux environment	Platform-specific build requirements apply
Node and workspace tools	Supported Node 22 LTS patch ≥22.12; pnpm 10.32.1; Turborepo	Record exact tested runtime/build versions
Web interface/backend	Next.js 16.2.0; React 19.2.4	Principal portal and backend workflows
Typing and styles	TypeScript 5.9.3; Tailwind CSS 4.2.2	Application implementation
Database	PostgreSQL; 16 proposed as isolated test baseline	Verify the deployed LMS version separately
Database access	Prisma 7.9.1 and PostgreSQL adapter/driver	Schema access and reviewed migrations
Mobile	Expo 55.0.31; React Native 0.83.10; Expo Router	Evaluate selected enabled functions
Desktop	Electron 41.0.3; Electron Builder	Native Administrator Console
Browsers	Current Chrome/Edge initially; Firefox/Safari where tested	Record exact versions and enabled activities
CodeLab execution	Configured Judge0-compatible endpoint	Verify service version, languages, isolation and limits
Repository coursework	Git and GitHub integration	Repository authorization is separate from LMS login
Files and email	Persistent object storage; configured account email service	Verify effective upload limits and supported delivery

Software/service	Recorded choice	Purpose/qualification
Testing	Playwright 1.63.0; TypeScript/Node and harness checks	Fresh functional, access and browser evidence
Observation and analysis	Developer tools, logs, database observations; Excel or LibreOffice Calc proposed	Select workload tools and record actual versions

Proposed evaluation targets are Windows 11 x64, Android 10+ and iOS 16+, conditional on verified builds.

3.8 FLOWCHART

The diagrams separate access and registration, role navigation, and course enrollment details. Oval terminators identify the start or end; rectangles identify processes; parallelograms identify input/output; diamonds identify decisions; and rectangles with double side lines identify detailed subprocesses. Circles continue a flow on the same page, while pentagonal connectors continue it on another diagram. Matching letters identify the destination. Each protected operation remains subject to backend authorization.

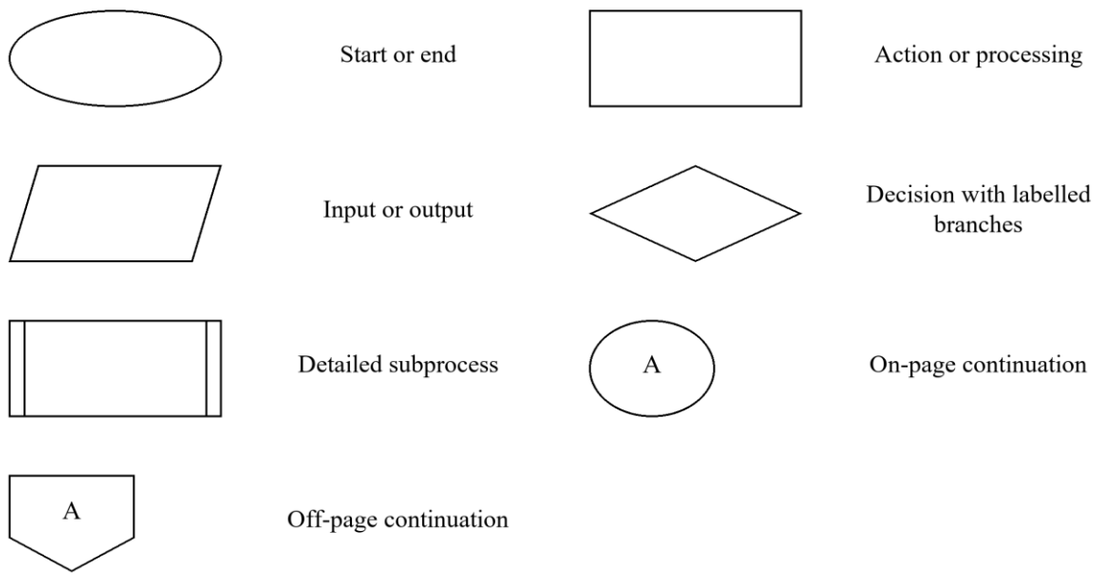


Figure 3
Symbols used in the system flowcharts

Off-page connectors R and L link registration with the access chart; S, I, A and U link to Student, Instructor, Administrator and Super User overviews. E and E2 lead to the enrollment request and Instructor review details. Same-page pairs L1, R1 and E1 allow correction and retry; S1, I1, A1 and U1 return to the corresponding dashboard. A dashed line marks the later review of a recorded pending request, which does not block the Student from using other modules.

3.8.1 Account Access and Registration

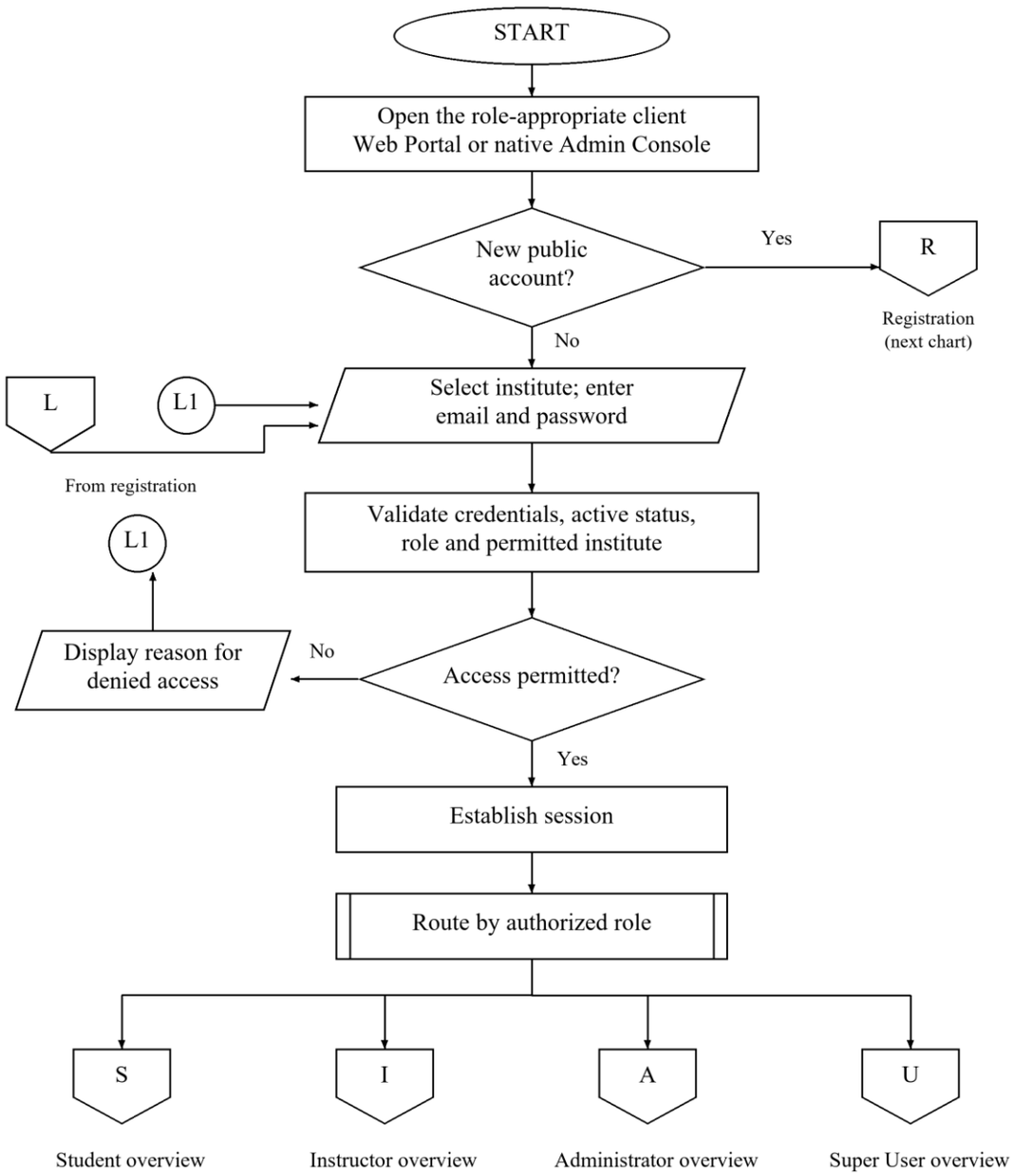


Figure 4

Account access and role routing

Users open the appropriate client and authenticate using email, password and institute selection. The backend validates credentials, account status, role and permitted institute before establishing a session. Invalid access produces feedback and a retry through L1. The target release routes successful access to the appropriate role overview. New public Student or Instructor accounts

use R; privileged accounts are created through authorized administrative operations. The Super User routing branch represents the agreed expansion and must be verified in the evaluation release.

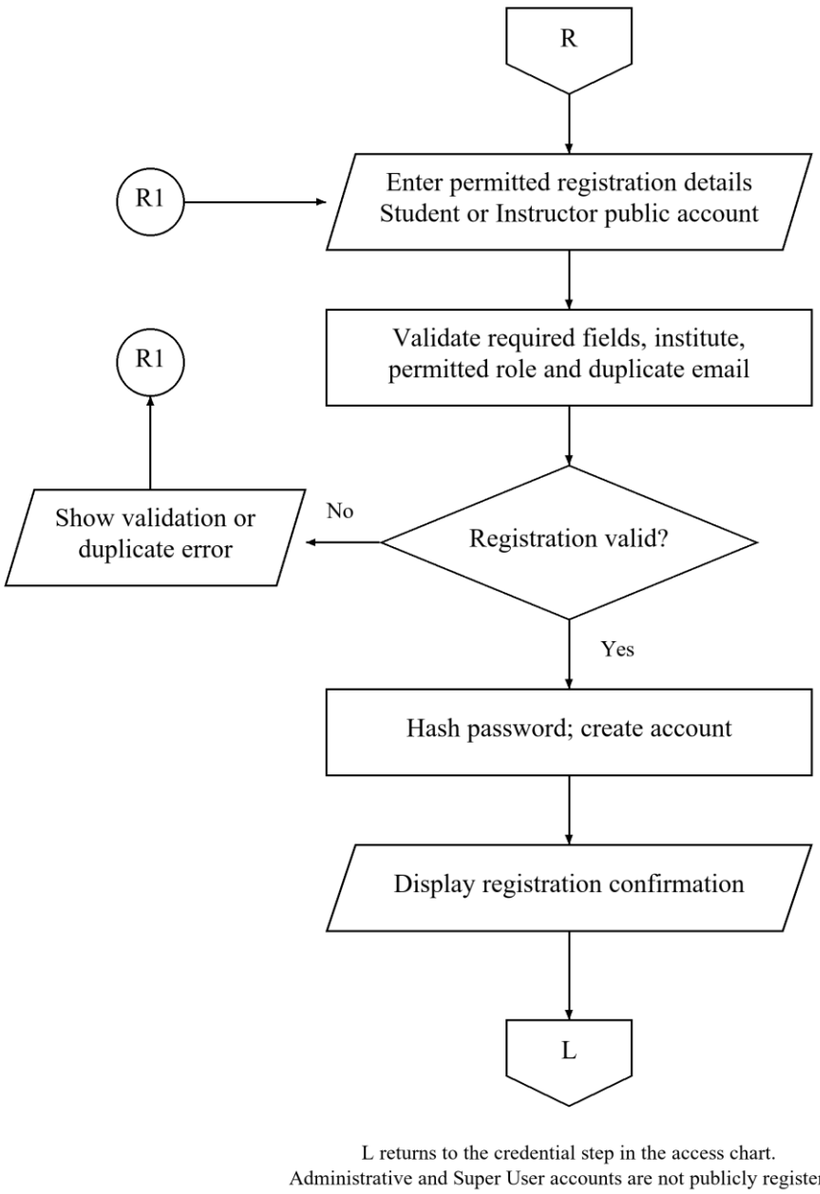


Figure 5

Public Student and Instructor registration detail

The registration process validates required fields, institute, permitted public role and duplicate email. Invalid input returns through R1 for correction. Valid registration hashes the password, creates the account, and displays confirmation. L returns to the access chart. Email verification or approval steps may be documented only when they exist in the evaluated registration workflow; no user-entered security pass-token is assumed.

3.8.2 Student View

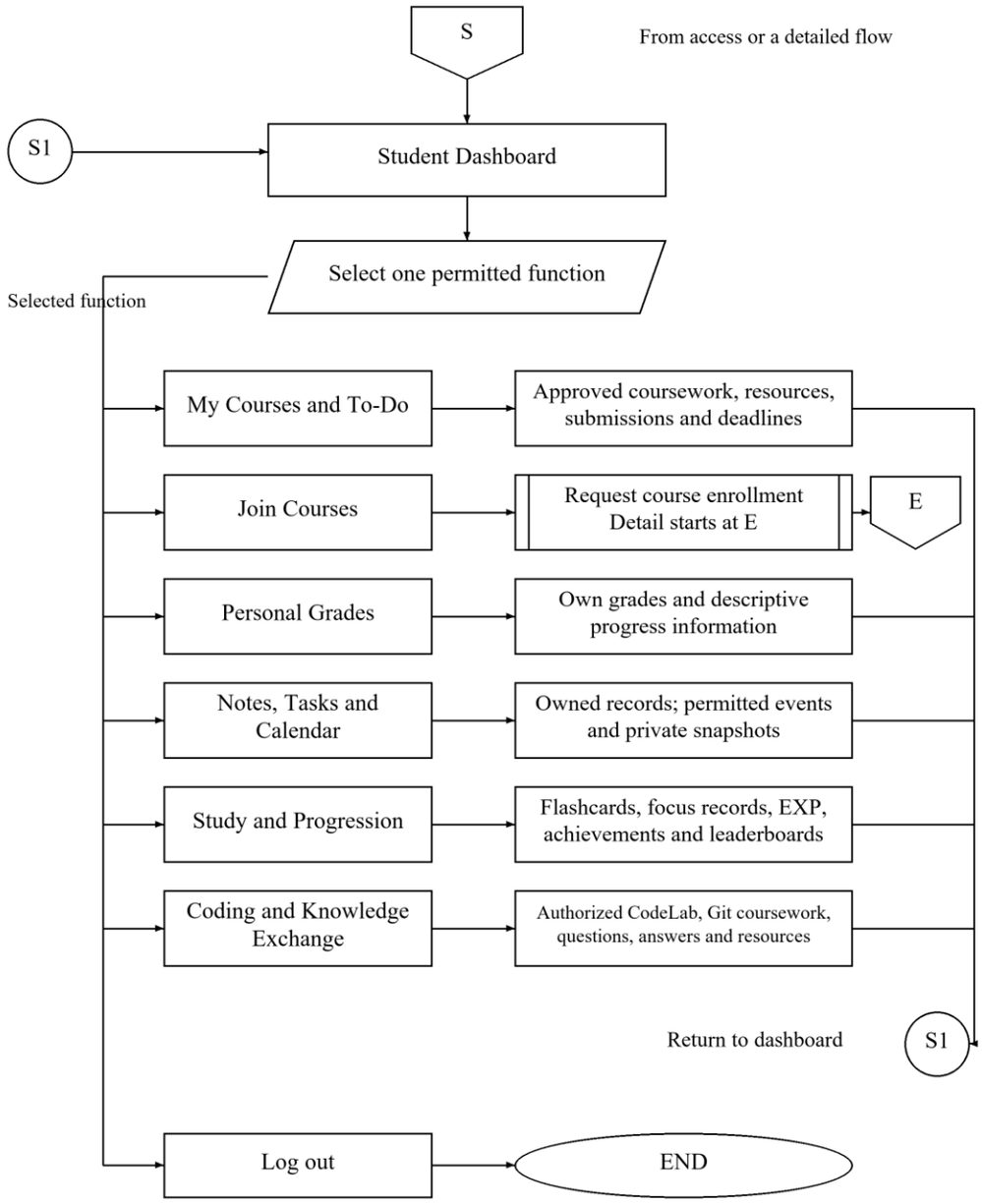
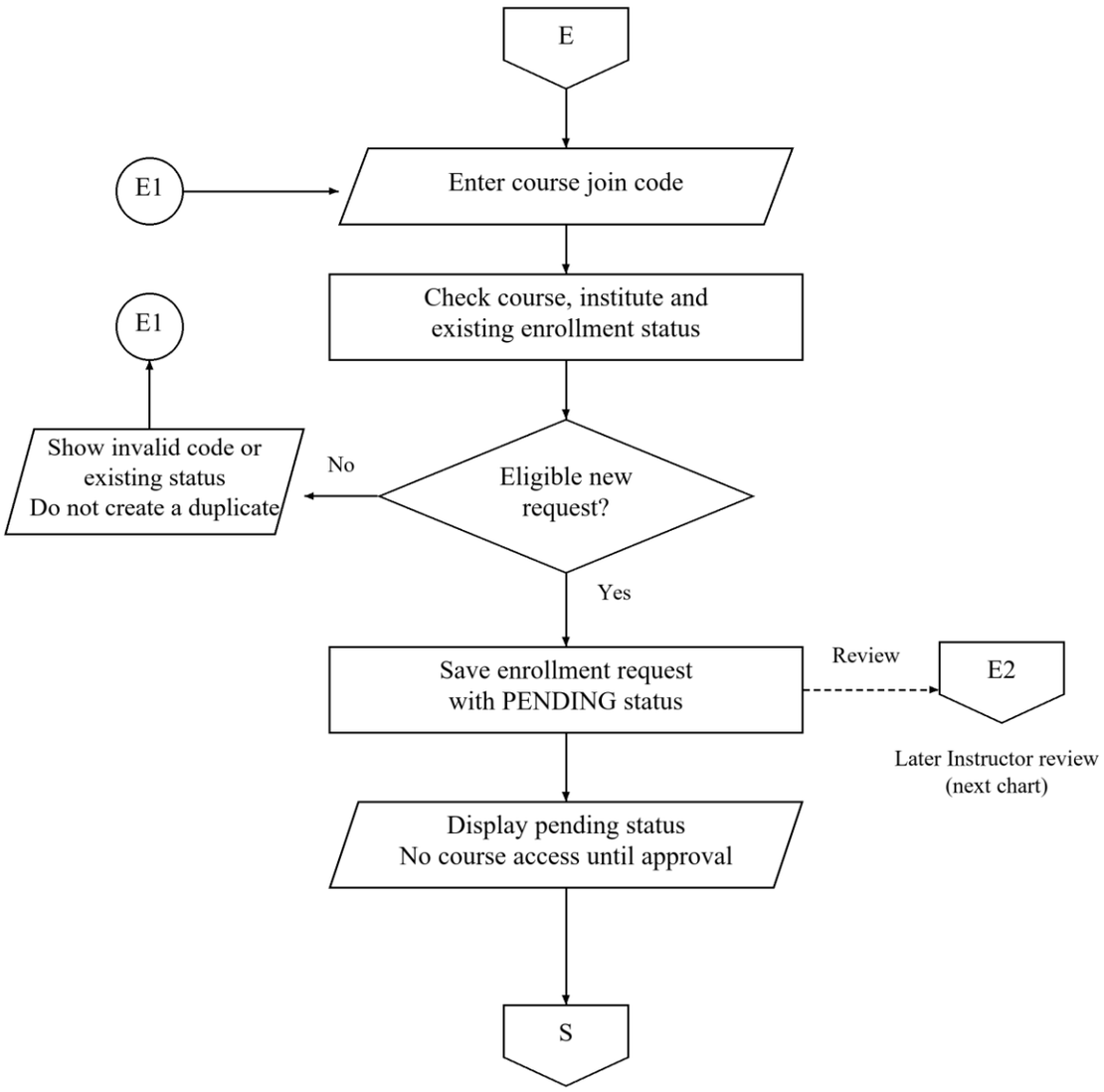


Figure 6

Student overview with links to detailed enrollment

S opens the Student Dashboard after successful access or a completed enrollment action. Students select approved coursework and assigned deadlines, request enrollment, view their own grades, and use permitted study, calendar, coding and collaboration tools. Personal Tasks remain separate from the academic To-Do list. Each ordinary module returns through S1; Join Courses continues through E. Students do not gain faculty-directory or supervisory privileges. Log out ends the session.



S returns to the Student overview. The Student can use other modules while waiting.

Figure 7

Student course enrollment request detail

E receives the Join Courses action. The backend checks the code, institute and existing enrollment status. Invalid or duplicate requests produce feedback and retry through E1. An eligible request is saved as PENDING, then shown to the Student. S returns to the Student overview without granting course access. E2 identifies the Instructor's later review of the saved request.

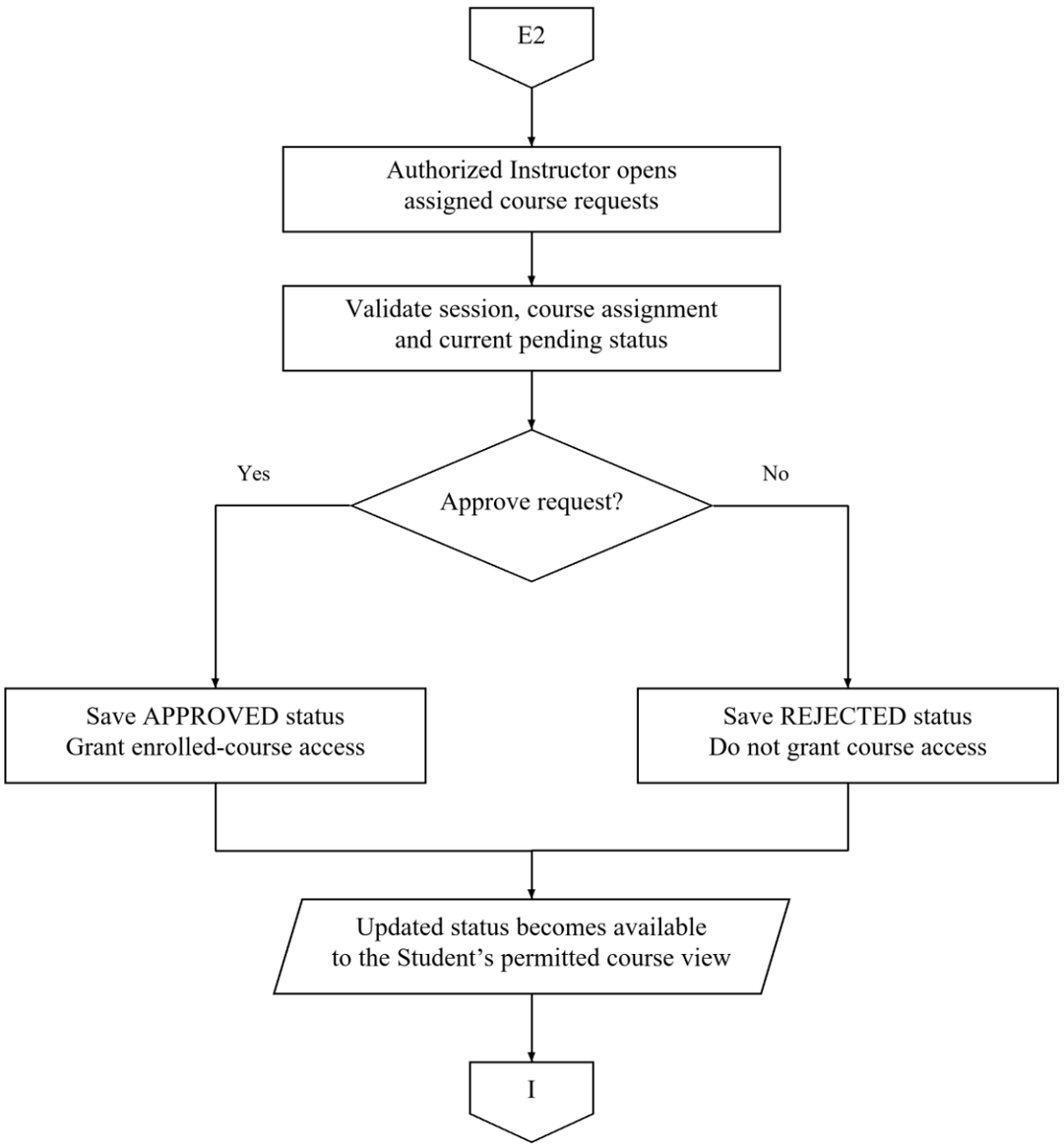


Figure 8

Instructor enrollment decision detail

The assigned Instructor opens the request queue and the backend checks authority and the current request state. Approval saves APPROVED and grants enrolled-course access; rejection saves REJECTED and does not grant access. Updated status becomes available through the Student's permitted course view. I returns to the Instructor overview. A valid course code alone does not enroll the Student immediately.

3.8.3 Instructor View

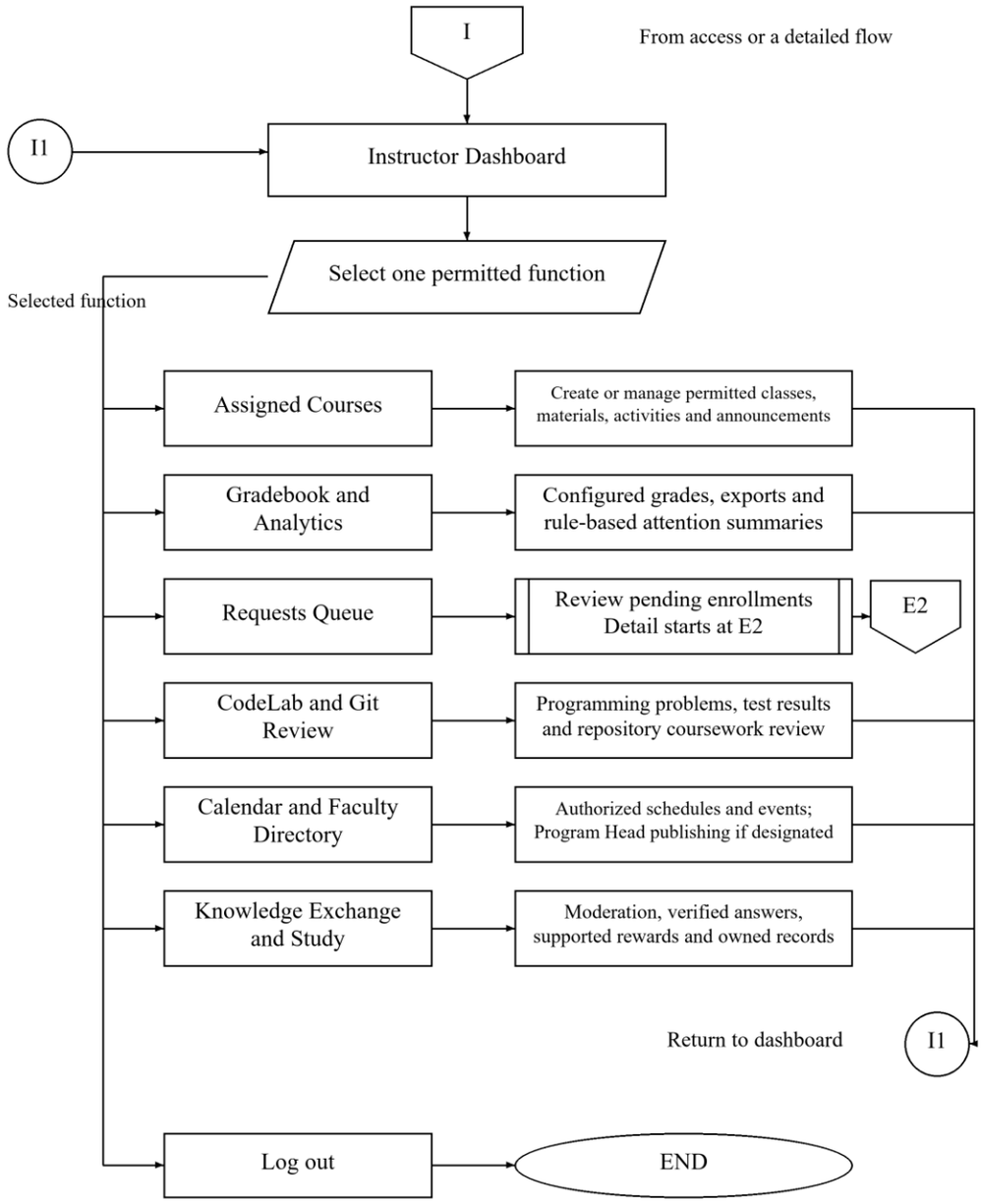


Figure 9

Instructor overview with enrollment review continuation

I opens the Instructor Dashboard. Instructors manage assigned courses and resources, configure grades, inspect descriptive analytics, review enrollment requests through E2, and assess CodeLab results or Git-linked work through their separate workflows. Calendar and faculty-directory actions follow their permissions; Program Head publishing requires that designation. Moderation, verification and owned study functions remain bounded by their rules. I1 returns to the dashboard, while Log out ends the session.

3.8.4 Administrator View

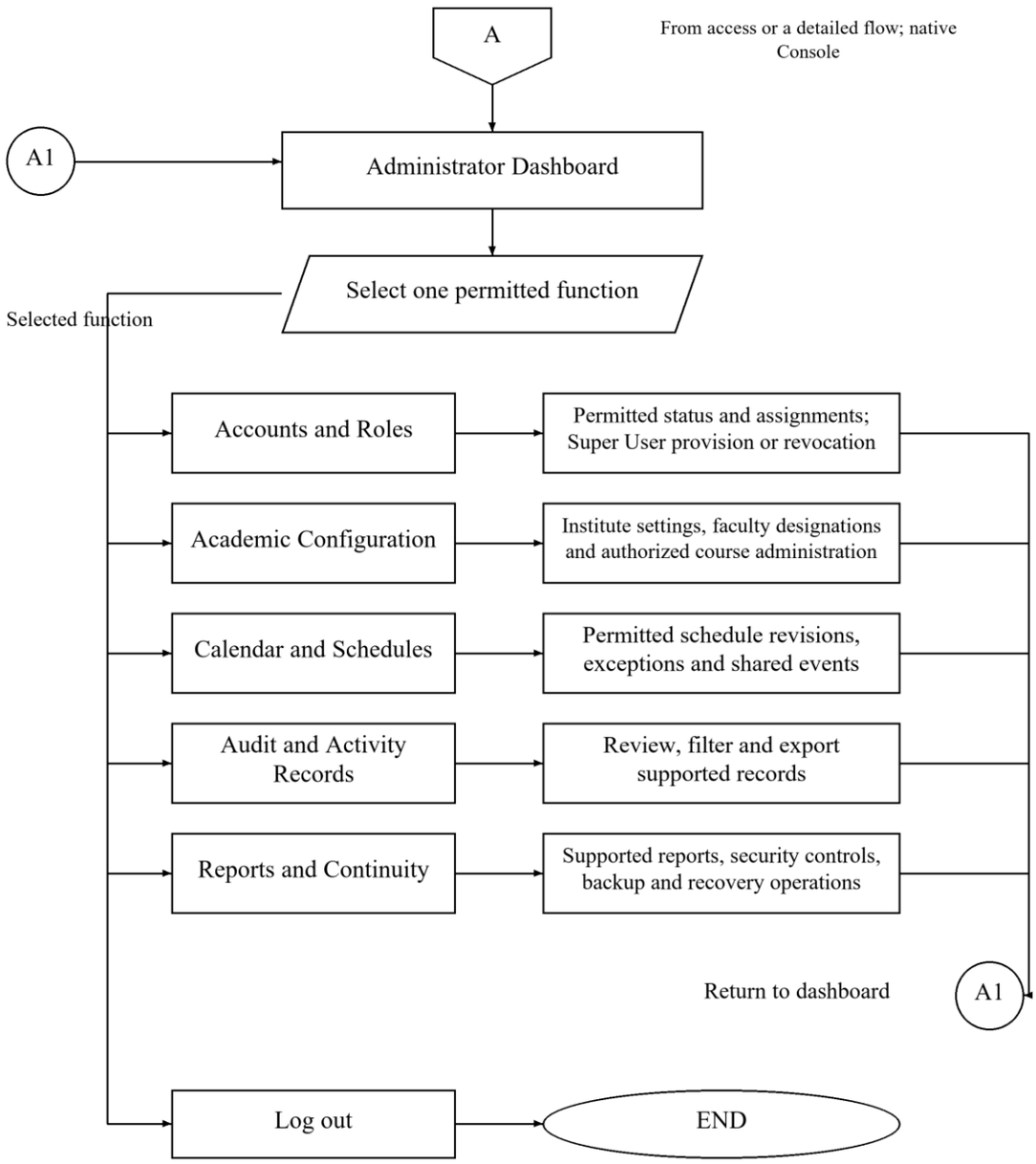


Figure 10

Native Administrator Console overview

A opens the native Administrator Console after validated Administrator access. Supported actions include accounts and roles, academic configuration, schedule/event administration, audit review, reports and continuity functions. Only authorized Administrators provision or revoke Super User accounts. Historical audit records are reviewed rather than freely rewritten. A1 returns to the dashboard. The researchers will verify these restricted functions with controlled accounts; no separate Administrator respondent group is introduced.

3.8.5 Super User View

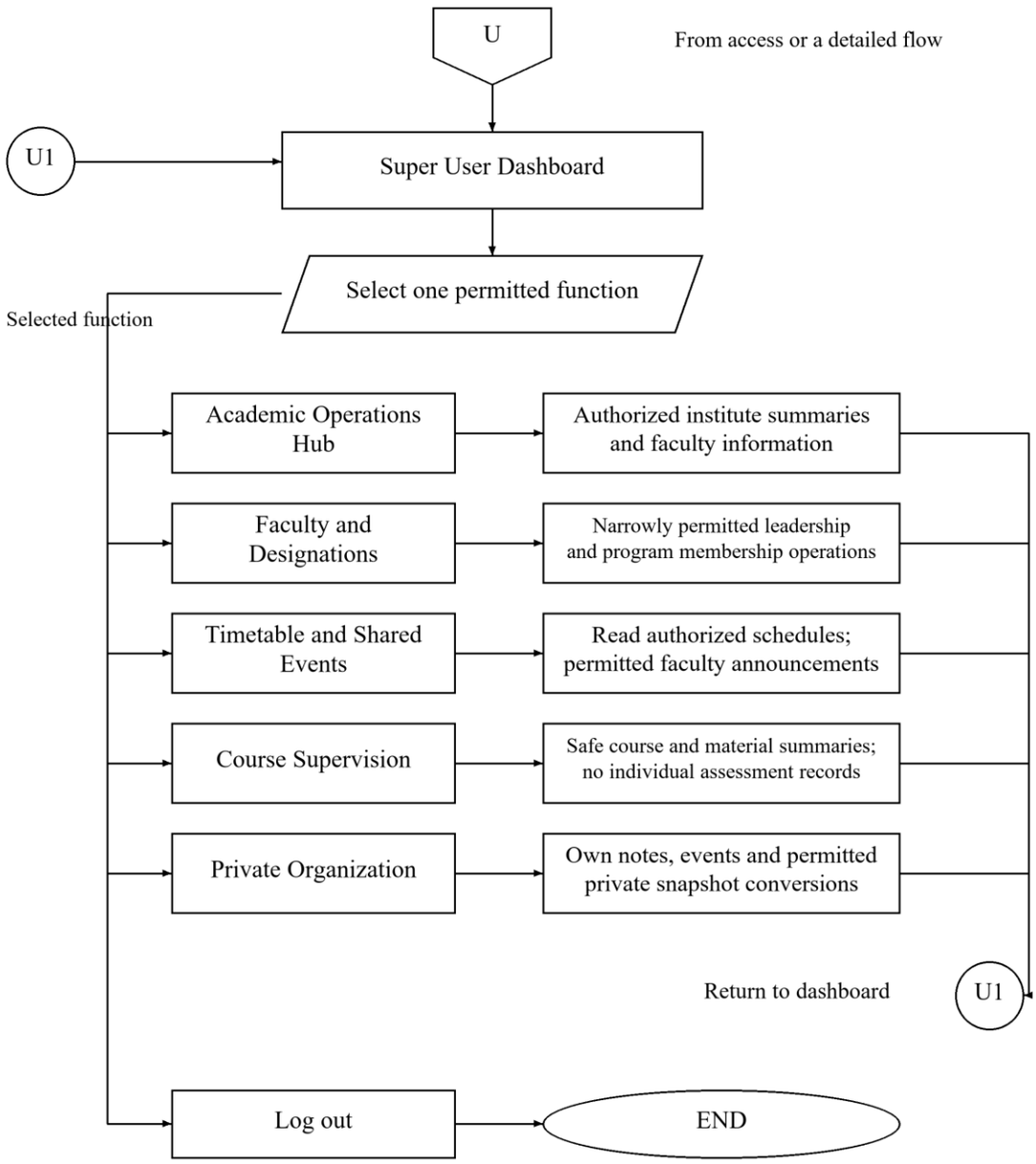


Figure 11

Restricted Super User academic operations overview

U opens the web-based academic operations hub for an authorized institute. The Super User uses permitted faculty and leadership information, reads timetables, publishes permitted shared events, inspects safe course summaries, and manages owned private records. Supervision excludes student identities, individual grades, submissions, attempts and answers. The role does not permit grading, assessment participation, schedule editing or generic administration. U1 returns to the hub; Log out ends the session. This expansion will undergo researcher-conducted technical checks.

3.9 ENTITY RELATIONSHIP DIAGRAM

The ERDs present selected relationships from the current Prisma schema and a separate target diagram for the coordination expansion. User stores account roles; Student, Instructor and Administrator are not independent account tables. Repeated entities across figures refer to the same logical table. PK denotes a primary key, FK a foreign key and UK a unique key or combination. Crow’s-foot notation shows multiplicity: a circle indicates zero, a bar indicates one, and a fork indicates many. Optional child collections may contain zero records.

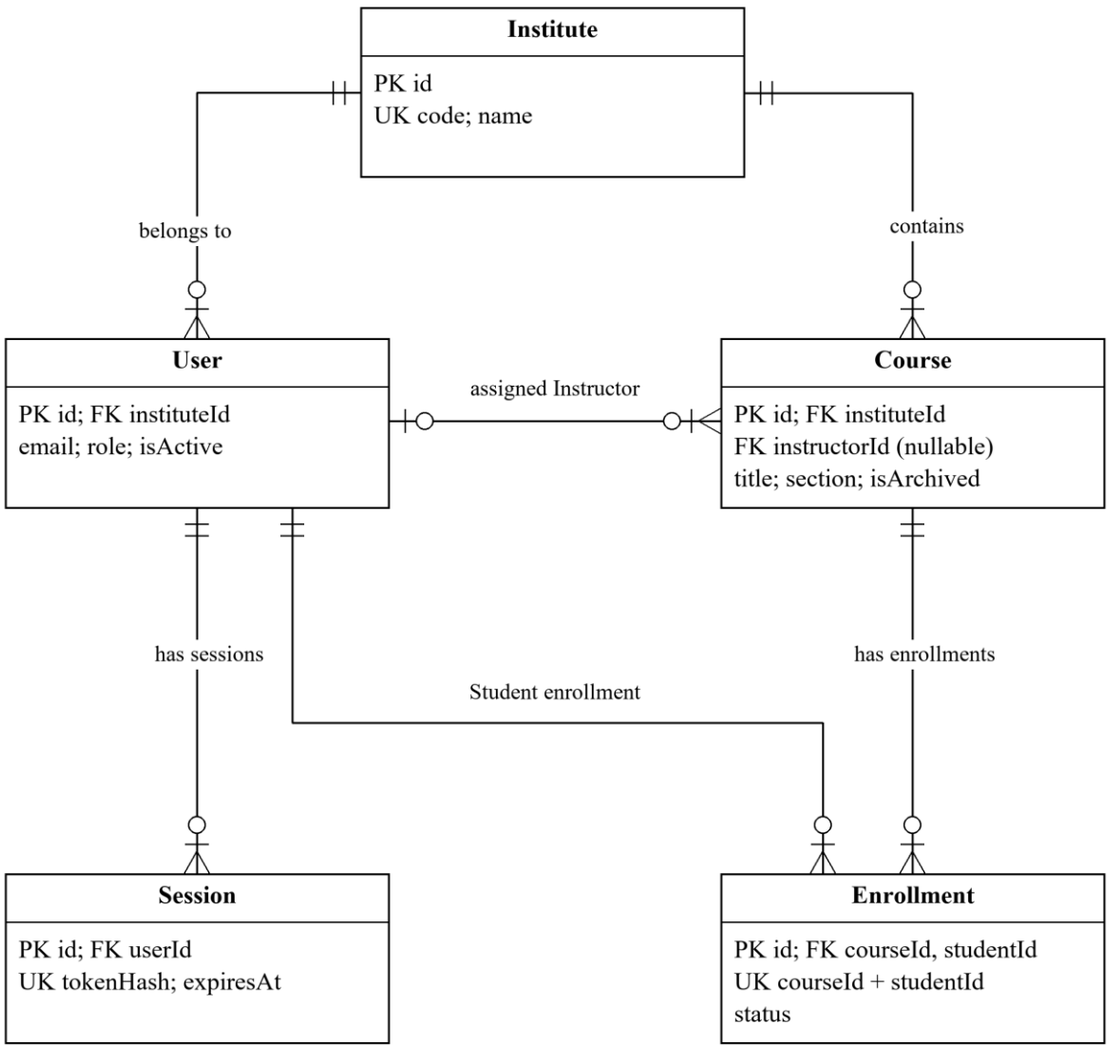


Figure 12

Account, institute and course enrollment relationships

Institute links User and Course records. Course.instructorId is optional and references a User acting as Instructor. Enrollment links a Student User to a Course and stores pending, approved or rejected status under a unique course/student pair. Session records belong to User. Role and resource authorization are enforced separately from these foreign-key links.

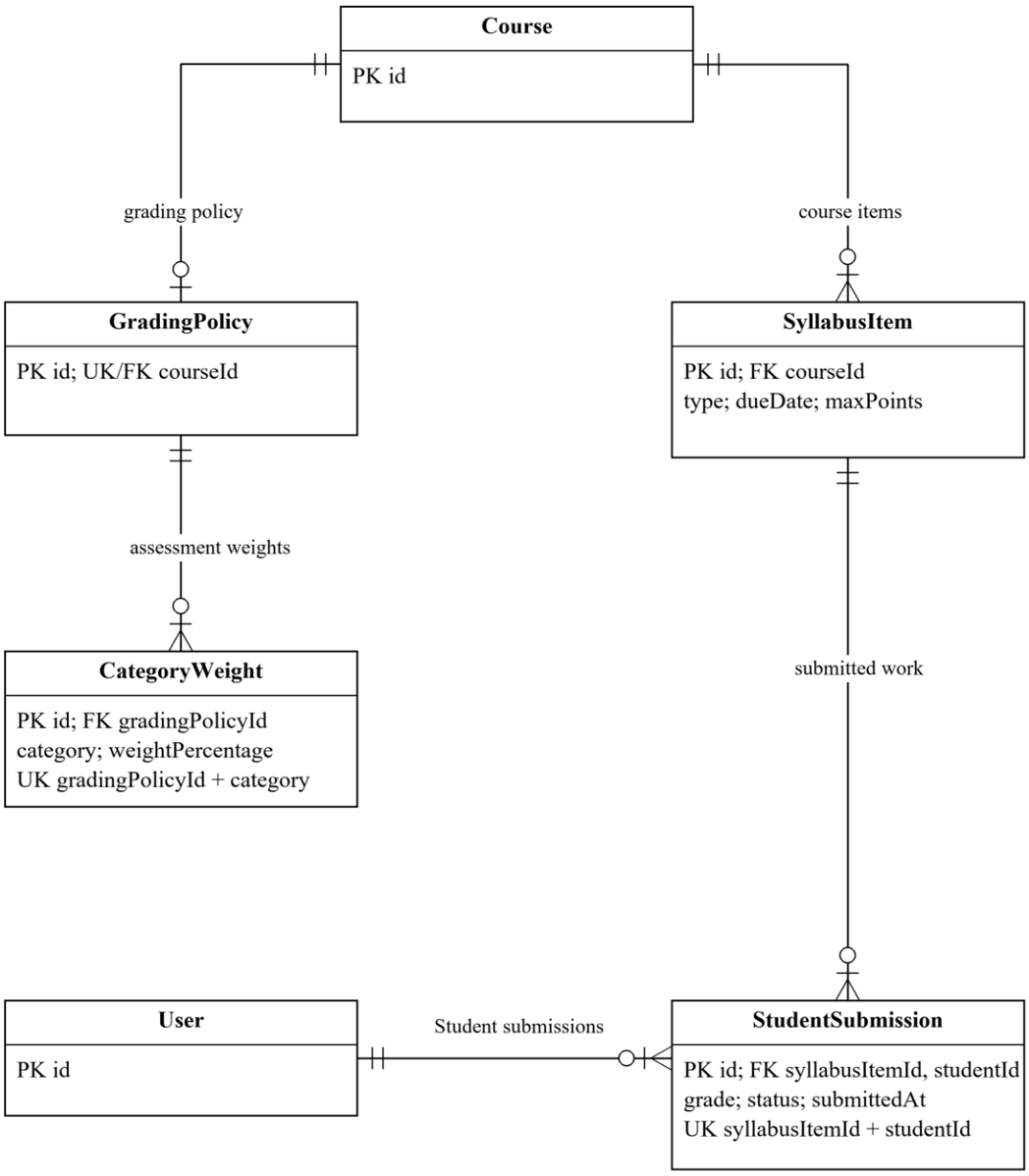


Figure 13

Coursework, submission and grading relationships

Course links SyllabusItem records and an optional GradingPolicy. CategoryWeight records define the policy's component weights. StudentSubmission connects an item to a Student User, with a unique item/student pair and recorded grade or status. Computed grade summaries use the applicable grading rules; they are not modelled as a single undifferentiated Grade table.

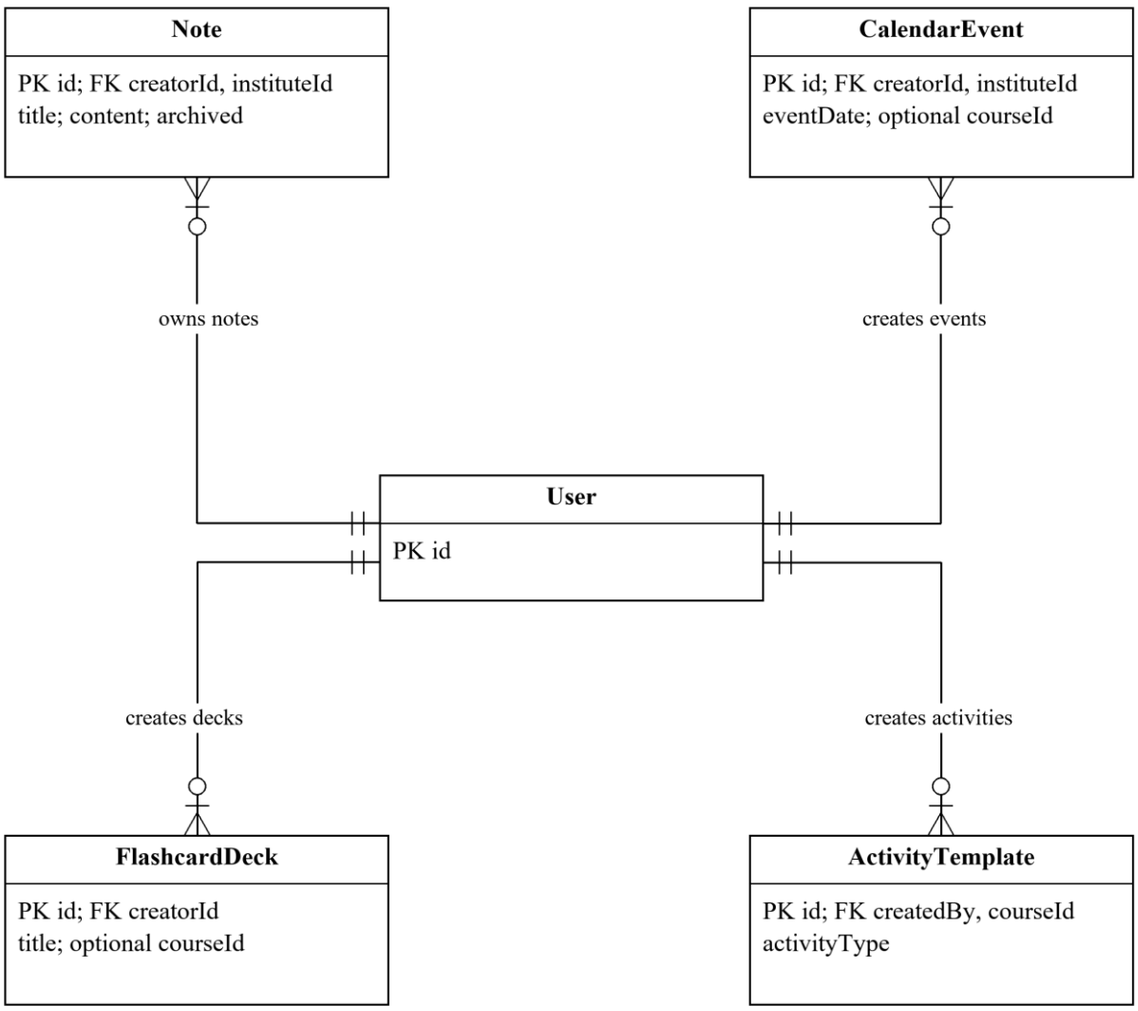


Figure 14

Selected personal ownership and learning activity relationships

User owns Note and CalendarEvent records, creates FlashcardDeck records, and creates ActivityTemplate records through createdBy. CalendarEvent may optionally reference Course, while ActivityTemplate requires a courseId. These selected relationships omit supporting task, card, progress, study-session and gamification tables for readability; their fields and constraints remain part of the release schema.

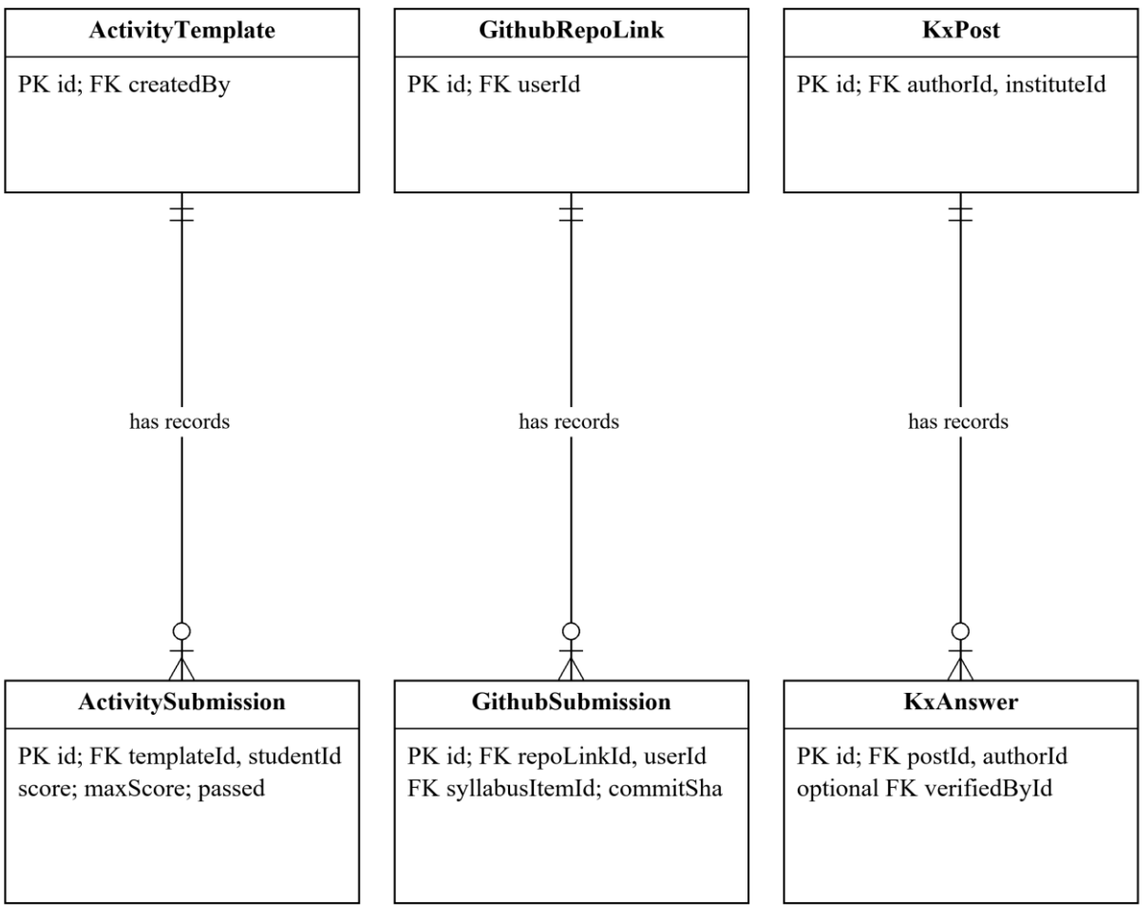


Figure 15

Separate programming, repository and forum record relationships

ActivityTemplate links ActivitySubmission records containing scores and execution-related outcomes. GithubRepoLink links GithubSubmission records, which also reference the submitting User and SyllabusItem. KxPost links KxAnswer records with an author and optional verifying faculty User. These groups preserve the distinction between code execution, Git coursework and moderated discussion.

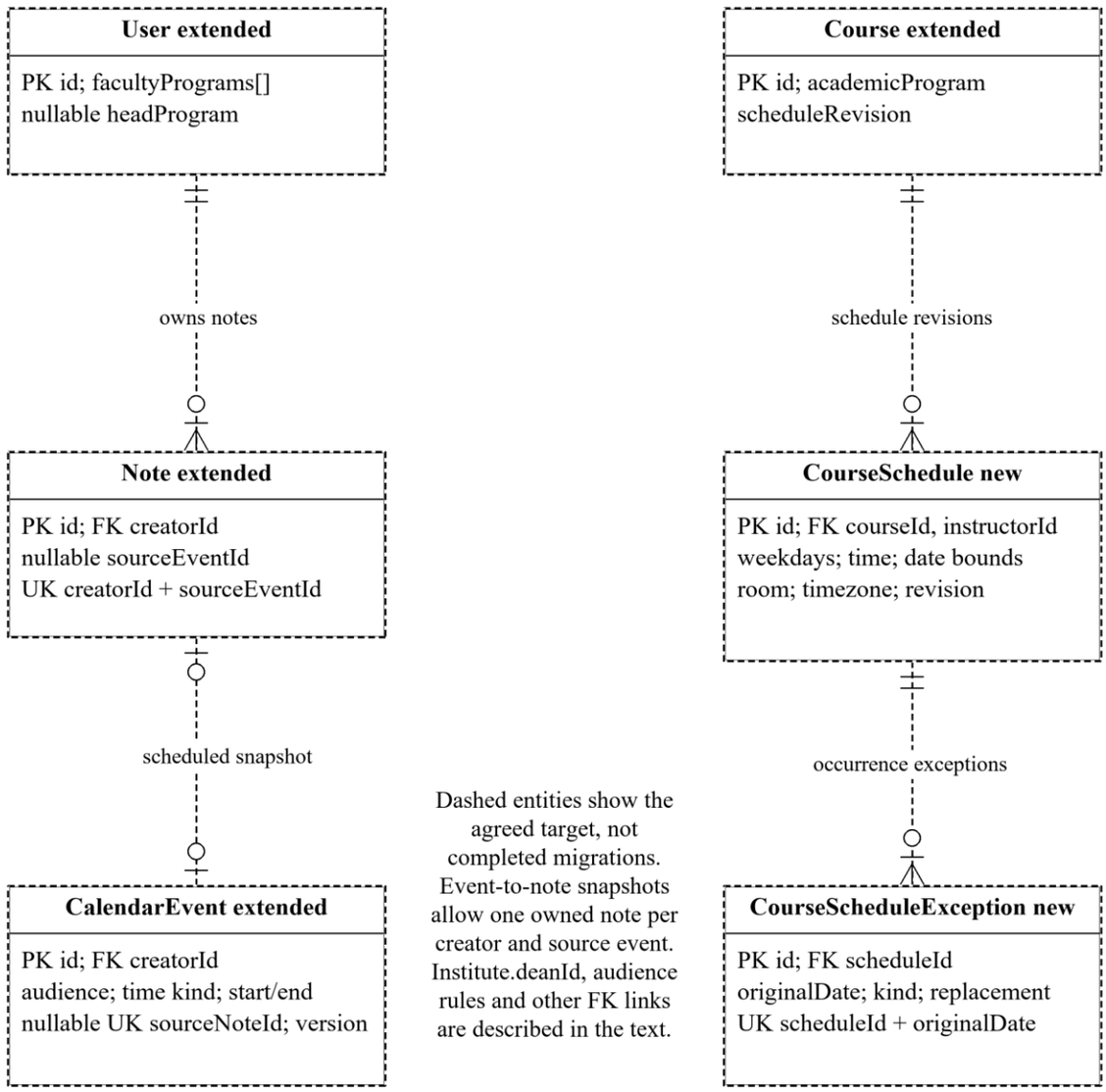


Figure 16

Proposed coordination schema and private snapshot relationships

Dashed outlines distinguish the target additions from current schema records. CourseSchedule and CourseScheduleException will preserve effective-dated revisions and occurrence exceptions. Course and User will receive program and coordination fields. CalendarEvent will receive time, audience, source-note and version fields; Note will receive an event-source link with a unique creator/event combination. Institute.deanId and other membership/attribution links will be checked against the final migration. Snapshot conversions are one-time copies, not ongoing synchronization.

3.10 DATA FLOW DIAGRAM

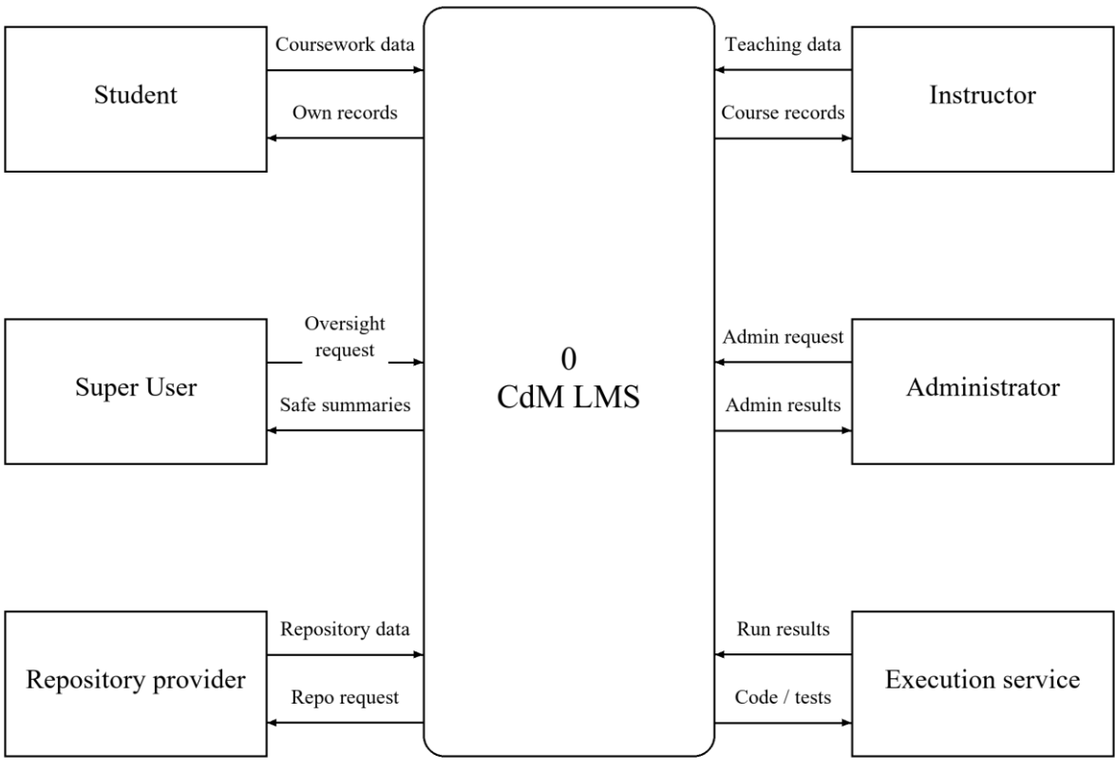


Figure 17

Context diagram of the LMS and external actors

The context diagram presents CdM LMS as one process exchanging data with Students, Instructors, Super Users, Administrators, the repository provider and the code-execution service. Rectangles represent external entities; the rounded process represents the system; and labelled arrows identify the direction of requests or returned information. Own records, safe summaries and administrative results remain limited by their respective permissions. Repository data and execution results are distinct exchanges. The role actors describe system users, not additional research respondent categories.

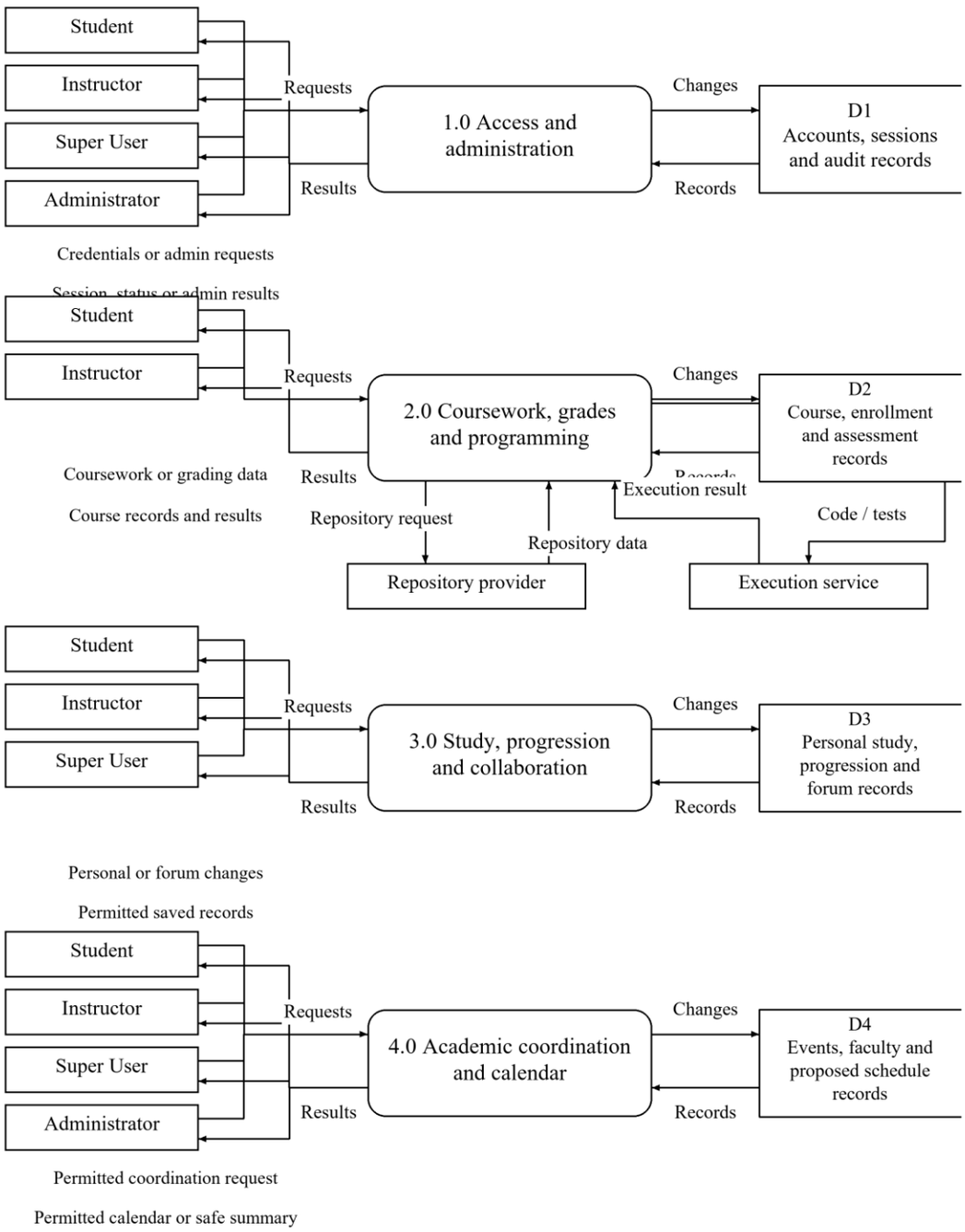


Figure 18

Level 1 functional data flow and logical record stores

Level 1 separates access/administration, coursework/grading/programming, study/progression/collaboration, and academic coordination/calendar processing. Repeated actor rectangles denote the same external entities shown in the context diagram. The request and result lines carry the data categories named beside their functional group. D1–D4 are logical record groups within the shared database, not additional tables. Each process enforces applicable session and

resource checks before reading or changing its permitted records. The repository and execution providers connect only to the corresponding programming workflow.

D4 combines existing event records with the proposed faculty and schedule sources. Generated recurring classes and syllabus deadlines are calendar outputs derived from authorized source records rather than routine persisted event rows. Student calendar reads exclude recurring timetable projection in the stated release. The final DFD and data-store descriptions will be reconciled with the enabled modules and frozen schema before evaluation.

REFERENCES

Abid, A., Tufail, A. H., Ahmed Salem, O. M., Nawaz, N. A., Farooq, U., Abid, I., & Ishaq, K. (2024). An evaluation framework and comparative analysis of the widely used learning management systems. *PLOS ONE, 19*(12), e0311111. https://doi.org/10.1371/journal.pone.0311111

Acala, J. J., & Dimasacat, A. C. B. (2023). Perceptions on the utilization of learning management system and its effects on the communication skills of private senior high school students in Manila. *Asian Journal of Education and Social Studies, 47*(3), 12–27. https://doi.org/10.9734/ajess/2023/v47i31024

Ahmed, H. M. M., El-Sabagh, H. A., & Elbourhamy, D. M. (2025). Effect of gamified, mobile, cloud-based learning management system (GMCLMS) on student engagement and achievement. *International Journal of Educational Technology in Higher Education, 22*, Article 49. https://doi.org/10.1186/s41239-025-00541-1

Alfalah, A. A. (2023). Factors influencing students' adoption and use of mobile learning management systems (m-LMSs): A quantitative study of Saudi Arabia. *International Journal of Information Management Data Insights, 3*(1), 100143. https://doi.org/10.1016/j.jjimei.2022.100143

Almodiel, M. C. (2022). Analyzing online learners' knowledge construction in asynchronous discussion forums using Interaction Analysis Model. *International Journal in Information Technology in Governance, Education and Business, 4*(1). https://doi.org/10.32664/ijitgeb.v4i1.93

Alsaqqa, S., Sawalha, S., & Abdel-Nabi, H. (2020). Agile software development: Methodologies and trends. *International Journal of Interactive Mobile Technologies, 14*(11), 246–270. https://doi.org/10.3991/ijim.v14i11.13269

Arada, B. P. (2025). Arduino programming education using Tinkercad: A mixed-method study on usability and student engagement during the COVID-19 pandemic. *International Journal in Information Technology in Governance, Education and Business, 7*(1), 135–150. https://doi.org/10.32664/ijitgeb.v7i1.175

Asuncion, A. E., Guadalupe, B. C., & Ortega, G. F. (2022). The ABC Workbook: Adapting online judge systems for introductory programming classes. In *Proceedings of the 30th International Conference on Computers in Education* (Vol. II, pp. 395–400). Asia-Pacific Society for Computers in Education. https://apsce.net/icce/icce2022/proceedings/

Bach, A., & Thiel, F. (2024). Collaborative online learning in higher education—quality of digital interaction and associations with individual and group-related factors. *Frontiers in Education, 9*, 1356271. https://doi.org/10.3389/feduc.2024.1356271

Benben, V. Y., & Bug-os, M. A. A. C. (2022). Physics students' academic achievement and motivation in a gamified formative assessment. *American Journal of Educational Research, 10*(6), 385–390. https://doi.org/10.12691/education-10-6-2

Çakiroğlu, Ü., Kokoç, M., & Atabay, M. (2024). Online learners' self-regulated learning skills regarding LMS interactions: A profiling study. *Journal of Computing in Higher Education, 36*, 220–241. https://doi.org/10.1007/s12528-024-09397-2

Dela Rosa, A. P. M. (2023). Exploring students' adoption of Canvas Learning Management System in programming at a state university in the Philippines: A technology acceptance perspective. *International Journal of Computing Sciences Research, 7*, 2287–2295. https://doi.org/10.25147/ijcsr.2017.001.1.161

Du, J., Liu, L., & Zhao, S. (2025). Empowering students in online learning environments through a self-regulated learning–enhanced learning management system. *Behavioral Sciences, 15*(8), 1041. https://doi.org/10.3390/bs15081041

Fernandez-Gauna, B., Rojo, N., & Graña, M. (2023). Automatic feedback and assessment of team-coding assignments in a DevOps context. *International Journal of Educational Technology in Higher Education, 20*, Article 17. https://doi.org/10.1186/s41239-023-00386-6

Fojas, A. F. C. (2022). *An empirical study evaluating the effects of gamification in an e-learning environment* [Capstone project, University of the Philippines Open University]. UPOU Repository. https://hdl.handle.net/20.500.13073/1294

Funa, A. A., Gabay, R. A. E., Deblois, E. C. B., Lerios, L. D., & Jetomo, F. G. J. (2023). Exploring Filipino preservice teachers' online self-regulated learning skills and strategies amid the COVID-19 pandemic. *Social Sciences & Humanities Open, 7*(1), 100470. https://doi.org/10.1016/j.ssaho.2023.100470

Guico, M. D. (2026). An analytical study on the utilization of learning management systems in higher education institutions: Perspectives from the Philippines. *Humanities and Social Sciences Letters, 14*(1), 126–140. https://doi.org/10.18488/73.v14i1.4668

Heikkinen, S., Saqr, M., Malmberg, J., & Tedre, M. (2023). Supporting self-regulated learning with learning analytics interventions—a systematic literature review. *Education and Information Technologies, 28*, 3059–3088. https://doi.org/10.1007/s10639-022-11281-4

International Organization for Standardization. (2023). *Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model* (ISO/IEC 25010:2023). https://www.iso.org/standard/78176.html

Jack, E. (2025). Exploring the impact of gamification in flipped classrooms through learning analytics [Conference abstract]. 65th ISI World Statistics Congress. https://www.isi-next.org/abstracts/submission/2543/view/

Jaramillo-Mediavilla, L., Basantes-Andrade, A., Cabezas-González, M., & Casillas-Martín, S. (2024). Impact of gamification on motivation and academic performance: A systematic review. *Education Sciences, 14*(6), 639. https://doi.org/10.3390/educsci14060639

Kumi, R. (2023). Discussion forums and student engagement: A social network analysis perspective. *Decision Sciences Journal of Innovative Education, 21*(4), 197–207. https://doi.org/10.1111/dsji.12299

Laspiñas, R. N., & Almerez, A. U. (2024). Users' experience study on the learning management system of a higher education institution in Cebu, Philippines. *Psychology and Education: A Multidisciplinary Journal, 17*(10), 1062–1066. https://doi.org/10.5281/zenodo.10827162

Llamas, M. C. A. R., Vilela-Malabanan, C. M., & Dinawanao, D. D. (2025). Needs and challenges of academics and students in using a learning management system: A user experience approach. *Journal of Education and Learning, 19*(3), 1662–1669. https://doi.org/10.11591/edulearn.v19i3.22350

McCombes, S. (2023, June 22). *Descriptive research: Definition, types, methods and examples*. Scribbr. https://www.scribbr.com/methodology/descriptive-research/

Memon, M. A., Thurasamy, R., Ting, H., & Cheah, J.-H. (2025). Purposive sampling: A review and guidelines for quantitative research. *Journal of Applied Structural Equation Modeling, 9*(1), 1–23. https://doi.org/10.47263/JASEM.9(1)01

Mennuti, B. (2024, April 17). *8 Google Classroom tips every teacher should know*. Google. https://blog.google/products-and-platforms/products/education/google-classroom-tips-teachers/

Messer, M., Brown, N. C. C., Kölling, M., & Shi, M. (2024). Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education, 24*(1), 1–43. https://doi.org/10.1145/3636515

Moodle. (2024, October 7). *Moodle LMS 4.5: Unlock the power of AI for your courses*. https://moodle.com/news/moodle-lms-45-unlock-the-power-of-ai-for-your-courses/

Mudau, P. K., & Van den Berg, G. (2023). Guidelines for supporting a community of inquiry through graded online discussion forums in higher education. *Education Sciences, 13*(9), 963. https://doi.org/10.3390/educsci13090963

Novak, M., & Kermek, D. (2024). Assessment automation of complex student programming assignments. *Education Sciences, 14*(1), 54. https://doi.org/10.3390/educsci14010054

Orvalho, P., Janota, M., & Manquinho, V. (2024). GitSEED: A Git-backed automated assessment tool for software engineering and programming education. In *Proceedings of the 2024 ACM Virtual Global Computing Education Conference* (Vol. 1, pp. 165–171). Association for Computing Machinery. https://doi.org/10.1145/3649165.3690106

Oyarzun, B., & Martin, F. (2023). A systematic review of research on online learner collaboration from 2012–21: Collaboration technologies, design, facilitation, and outcomes. *Online Learning, 27*(1), 71–106. https://doi.org/10.24059/olj.v27i1.3407

Pan, S. C., Zung, I., Imundo, M. N., Zhang, X., & Qiu, Y. (2022). User-generated digital flashcards yield better learning than premade flashcards. *Journal of Applied Research in Memory and Cognition*. Advance online publication. https://doi.org/10.1037/mac0000083

Patani, P., Tiwari, S., & Rathore, S. S. (2024). The impact of GitHub on students' learning and engagement in a software engineering course. *Computer Applications in Engineering Education, 32*, e22775. https://doi.org/10.1002/cae.22775

Peralta, R. L. (2024). Development and evaluation of course management system with short messaging service notification for Centro Escolar University. *GEO Academic Journal, 5*(1). https://doi.org/10.56738/issn29603986.geo2024.5.46

Purwanto, Y. S., Gernowo, R., & Nugraheni, D. M. K. (2025). Adapting IMOI for information systems: Toward a cyclical and mediated view of system design. *PETIR*, *18*(1), 1–12. https://jurnal.itpln.ac.id/petir/article/view/2869

Quimod, M. T., Alegata, W. C. G., & Madrigal, D. V. (2023). e-Modyul: Technical-vocational-livelihood support learning management system. *Technium Education and Humanities, 5*, 9–26. https://doi.org/10.47577/teh.v5i.8890

Ramos, R. C., Garcia, P., Roxas-Ridulme, Q., Cabanes, R. V., Rosario, H. M., Enrile, M. K., De Jesus, R., Dolliente, C., & Sandoval, A. (2024). Development of a Filipinized learning management system. *SN Computer Science, 5*, Article 275. https://doi.org/10.1007/s42979-024-02621-x

Schauer, L., Stewart, R. J., & Maarek, M. (2024). Integrating Canvas and GitLab to enrich learning processes. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training* (pp. 180–190). Association for Computing Machinery. https://doi.org/10.1145/3639474.3640056